

# APPROXIMATION ALGORITHMS FOR BIN-PACKING – AN UPDATED SURVEY

E.G. Coffman, Jr.

M.R. Garey

D.S. Johnson

Bell Laboratories

Murray Hill, New Jersey 07974

## 1. Introduction

This paper updates a survey [53] written about 3 years ago. All of the results mentioned there are covered here as well. However, as a major justification for this second edition we shall be presenting many new results, some of which represent important advances. As a measure of the impressive amount of research in just 3 years, the present reference list more than doubles the list in [53].

Characteristic of bin-packing applications is the necessity to pack or fit a collection of objects into well-defined regions so that they do not overlap. From an engineering point of view the problem is normally one of making efficient use of time and/or space. A basic mathematical model is defined in the classical one-dimensional bin packing problem: We are given a positive integer bin capacity  $C$  and a set or *list* of items  $L = (p_1, p_2, \dots, p_n)$ , each item  $p_i$  having an integer size  $s(p_i)$  satisfying  $0 \leq s(p_i) \leq C$ . What is the smallest integer  $m$  such that there is a partition  $L = B_1 \cup B_2 \cup \dots \cup B_m$  satisfying  $\sum_{p_i \in B_j} s(p_i) \leq C$ ,  $1 \leq j \leq m$ ? We usually think of each set  $B_i$  as being the contents of a *bin* of capacity  $C$ , and view ourselves as attempting to minimize

the number of bins needed for a packing of  $L$ .

By this choice of terms the obvious interpretation of bin-packing corresponds to problems of storage. However, the variety of other interpretations that can be placed on the parameters and terminology accounts for the fundamental importance of the problem. Packing trucks with a given weight limit and assigning commercials to station breaks on television [11] illustrate this variety in the real world. A commonly cited, general example is the following cutting-stock problem. Material such as cable, lumber, pipes, tapes, etc. is supplied in a standard length,  $C$ . Demands for pieces of the material are for arbitrary lengths not exceeding  $C$ . The problem is to use the minimum number of standard lengths in accommodating a given list of required pieces.

Problems in which time is the dimension (resource) being partitioned are represented by the following scheduling problem: We are given a collection of identical processors on which a set of independent tasks with known execution times are to be executed. The problem is to determine the least number of processors that must be used in order that all tasks be completed by some given deadline. Here, the processors are bins, the deadline is the common bin capacity and the elements of  $L$  are the task execution times.

This problem establishes the connection between bin-packing and combinatorial scheduling theory. Note in particular the close relationship between this problem and the multiprocessor scheduling problem, i.e. the problem of minimizing makespan on parallel processors. (The makespan or length of a schedule is simply the latest task finishing time.) In bin-packing terms it corresponds to the bin design problem: Given  $L$  and a fixed set of  $m$  bins, what is the least capacity  $C$  such that  $L$  can be packed in  $m$  bins of this capacity? Historically, research on this problem was carried out exclusively under the heading of scheduling theory. These results are included here because *both* the classical and capacity minimization problems have significance in scheduling and storage applications, and because they are based on the same mathematical structure, i.e. they differ

only in the descriptor held fixed and the one chosen as the objective function.

It is consistent with the effort invested in these problems that efficiently computing optimal solutions has proved to be quite difficult. In fact, the bin-packing problem, or more precisely the decision problem "Given  $C$ ,  $L$ , and an integer bound  $K$ , can  $L$  be packed into  $K$  or fewer bins of capacity  $C$ ?" is *NP*-complete. A similar statement holds for the decision problem corresponding to multiprocessor scheduling. By the theory elaborated in [52,71,78], this means that it is unlikely that efficient, (i.e., polynomial time) optimization algorithms can be found for these problems. Thus researchers have turned to the study of approximation algorithms, that is, algorithms which, although not guaranteed to find an optimal solution for every instance, attempt to find *near-optimal* solutions. The analysis of approximation algorithms is the dominant topic in the remainder of this paper. It is primarily this theme that has determined the literature we have chosen to survey. This theme will be further narrowed to those relatively simple but effective algorithms which have been successfully analyzed for measures of worst-case or average-case performance.

Along with closely related partitioning problems, bin-packing and multiprocessor scheduling have played an important role in applications of complexity theory [52]. They also hold a special place in the history of approximation algorithms. It was in these contexts that the first work was done in proving that fast approximation algorithms could actually *guarantee* near-optimal solutions. The early work of Graham [60,62] on multiprocessor scheduling inaugurated this approach, and the early work in bin packing (most notably [72]) served to popularize and extend the methodology.

The scope of applications has been widened considerably by the study of a number of variants of the basic problem. Approximation algorithms have been designed and analyzed for the following four basic modifications: (1) Packings in which bounds are placed in advance on the number of items that can be packed in a bin, (2) Packings in which a partial order is associated with the set of items to be packed and constrains the ways in which items may be packed. (3) Packings in which

restrictions are placed on the items that may be packed in the same bin, and (4) Packings in which items may enter and leave the packing dynamically. These variants will be covered in Section 3 following the survey of results in Section 2 for the classical problem.

Multiprocessor scheduling results will be described in Section 4. In addition to this problem, there have been several others based on optimizing objective functions other than the number of bins. In Section 5 we shall consider such objective functions as the number of items packed and the sum of the squares of the bin levels, where bin level refers to the total size of the items in a bin.

With the techniques that had developed for the one-dimensional problems it was natural that efforts eventually turn to higher dimensions. In Section 6 we discuss the results on vector packing where items sizes and bin capacity are assumed to be  $d$ -dimensional vectors. Such problems model scheduling applications in which jobs must use several different resources during their execution.

In Section 7 we survey the large and growing literature on two dimensional packing. Once again the obvious industrial applications in stock cutting have been an important stimulus to this research. Further motivation has been provided by advances in VLSI technology in which layouts on chips pose a number of important combinatorial packing problems. The focus of this survey on the analysis of approximation algorithms essentially limits us to the research on packing rectangular figures into two dimensional "bins" or strips.

We shall be covering numerous improvements to the early results in bin-packing that were not mentioned in [53]. This includes the discovery of polynomial approximation schemes for one-dimensional bin packing, and the many new results on the probabilistic analysis of packing algorithms. In the final section we shall mention a few of the many open problems still outstanding.

## **2. The Classical Bin-Packing Problem**

We begin by describing three basic algorithms for the problem as defined in the preceding

section. The first, and simplest, is NEXT-FIT: We process the items in  $L$  in turn, starting with  $p_1$ , which is placed in bin  $B_1$ . Suppose that  $p_i$  is now to be packed, and let  $B_j$  be the highest indexed non-empty bin. If  $p_i$  will fit in  $B_j$  (the level of  $B_j$  does not exceed  $C - s(p_i)$ ), then put  $p_i$  in bin  $B_j$ . Otherwise, start a new bin (bin  $B_{j+1}$ ) by putting  $p_i$  into it.

This is clearly a fast algorithm (linear time). Moreover, it is not difficult to show that, if  $NF(L)$  is the number of bins used in the NEXT FIT packing of list  $L$  and  $OPT(L)$  is the number of bins required in an optimal packing, then for all lists  $L$ ,  $NF(L) \leq 2 \cdot OPT(L)$ . This is the best bound of this sort we can prove for NEXT FIT, since the examples shown in Figure 1 indicate that there are lists  $L$  with  $NF(L) \geq 2 \cdot OPT(L) - 1$ .

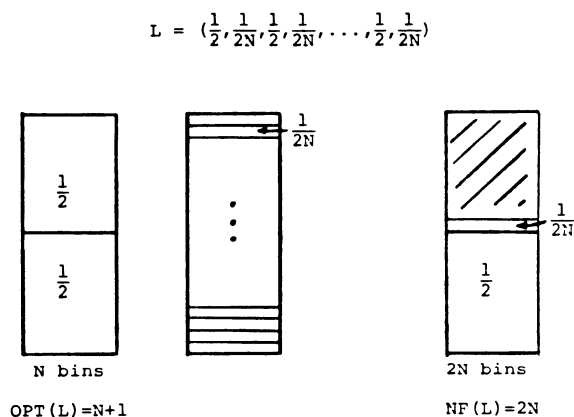


Figure 1. Examples of lists  $L$  with  $NF(L) = 2 \cdot OPT(L) - 1$ .

To improve on this bound we need a new algorithm. One defect of NEXT FIT seems to be that it only tries to put  $p_i$  in one bin before it resorts to starting a new bin. This suggests that the following FIRST FIT algorithm might be an improvement: When packing  $p_i$ , put it in the lowest indexed bin into which it will fit (starting a new bin only if  $p_i$  will not fit into any non-empty bin). It can be shown (though the proof [49,72] is more difficult) that for all lists  $L$ ,

$FF(L) \leq (17/10) \cdot OPT(L) + 1$  [72] and, again, this is the best ratio possible, since there are lists  $L$  with arbitrarily large values of  $OPT(L)$  such that  $FF(L) \geq (17/10) \cdot OPT(L) - 8$  [72]. These lists are too complicated to illustrate here, but Figure 2 shows examples that approach a ratio of  $5/3 = 1.6666\dots$

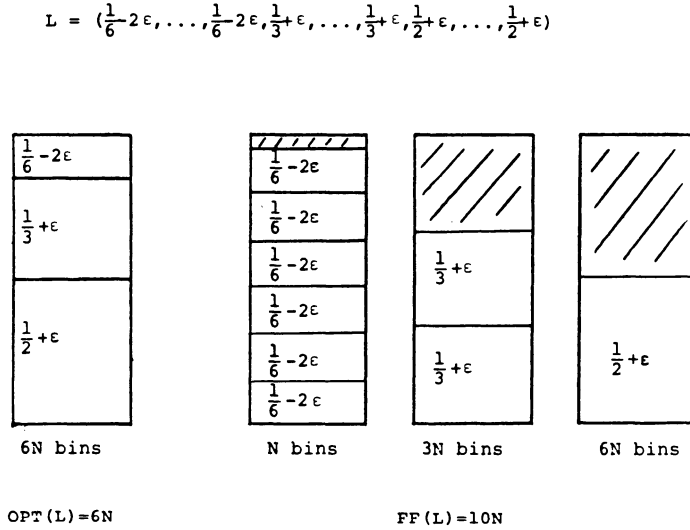


Figure 2. Examples of lists  $L$  with  $FF(L) = \frac{5}{3} \cdot OPT(L)$ .

From these examples a further improvement suggests itself. FIRST FIT seems to perform poorly when the large items occur at the end of the list. The algorithm FIRST FIT DECREASING seeks to avoid this effect by first ordering the items so that  $s(p_1) \geq s(p_2) \geq \dots \geq s(p_n)$ , and then applying FIRST FIT to the reordered list. For this algorithm it can be shown (with considerable difficulty [4,69,72]) that for all lists  $L$ ,  $FFD(L) \leq \frac{11}{9} \cdot OPT(L) + 4$  and, once more, this is the best ratio possible, as illustrated in Figure 3.

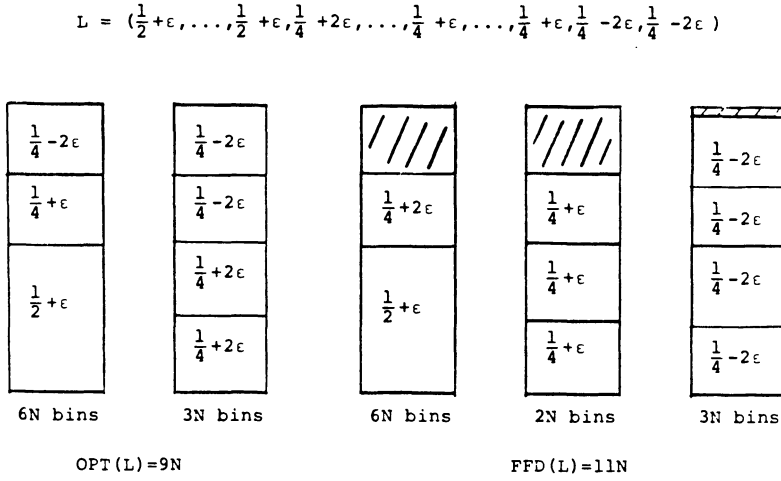


Figure 3. Examples of lists  $L$  with  $FFD(L) = \frac{11}{9}OPT(L)$ .

Let us formalize the type of worst case analysis we have been discussing. If  $A$  is an algorithm and  $A(L)$  is the number of bins used by that algorithm for list  $L$ , define  $R_A(L) \equiv A(L)/OPT(L)$ . The *absolute performance ratio*  $R_A$  for algorithm  $A$  is given by

$$R_A \equiv \inf\{r \geq 1: R_A(L) \leq r \text{ for all lists } L\}.$$

The *asymptotic performance ratio*  $R_A^\infty$  for  $A$  is given by

$$R_A^\infty \equiv \inf\{r \geq 1: \text{for some } N > 0, R_A(L) \leq r \text{ for all } L \text{ with } OPT(L) \geq N\}.$$

The above results can now be summarized by saying that  $R_{NF}^\infty = 2$ ,  $R_{FF}^\infty = 17/10$ , and  $R_{FFD}^\infty = 11/9$ . Notice that  $R_A$  need not equal  $R_A^\infty$ . Although  $R_{FFD}^\infty = 11/9$ , it is easy to give lists  $L$  for which  $OPT(L) = 2$  and  $FFD(L) = 3$ , so that  $R_{FFD} \geq 3/2$ . The asymptotic ratios seem to be a more reasonable measure of performance for the basic bin packing problem, but absolute ratios do come up in some of the work on related problems that we shall be discussing later.

Table 1 highlights the early results that were obtained for several other algorithms, along with

those just described. The quantity  $R_A^\infty(t)$ ,  $0 < t \leq 1$  is the asymptotic worst case ratio for algorithm  $A$  on lists all of whose items have size bounded by  $t \cdot C$ . This measure is of interest in applications where the largest item expected is significantly smaller than the bin capacity.

| Algorithm         | Timing               | $R_A^\infty$ | $R_A^\infty(1/2)$ | $R_A^\infty(1/3)$ | $R_A^\infty(1/4)$ |
|-------------------|----------------------|--------------|-------------------|-------------------|-------------------|
| WORST FIT         | $\theta(n \log n)$   | 2.0          | 2.0               | 1.5               | 1.333...          |
| NEXT FIT          | $\theta(n)$          | 2.0          | 2.0               | 1.5               | 1.333...          |
| FIRST FIT         | $\theta(n \log n)$   | 1.7          | 1.5               | 1.333...          | 1.25              |
| BEST FIT          | $\theta(n \log n)$   | 1.7          | 1.5               | 1.333...          | 1.25              |
| ALMOST WORST FIT  | $\theta(n \log n)$   | 1.7          | 1.5               | 1.333...          | 1.25              |
| NF DECREASING     | $\theta(n \log n)$   | 1.691...     | 1.424...          | 1.302...          | 1.234...          |
| REVISED FF        | $\theta(n \log n)$   | 1.666...     | NA                | NA                | NA                |
| GROUP FIT GROUPED | $\theta(n)$          | 1.5          | 1.333...          | 1.25              | 1.20              |
| FF GROUPED        | $\theta(n \log n)$   | 1.333...     | 1.333...          | 1.25              | 1.20              |
| ITERATED LFD      | $\theta(n \log^2 n)$ | 1.333...     | NA                | NA                | NA                |
| FF DECREASING     | $\theta(n \log n)$   | 1.222...     | 1.183...          | 1.183...          | 1.15              |
| BF DECREASING     | $\theta(n \log n)$   | 1.222...     | 1.183...          | 1.183...          | 1.15              |
| MODIFIED FFD      | $\theta(n \log n)$   | 1.183...     | 1.183...          | 1.183...          | 1.15              |

**Table 1.** Asymptotic worst case bounds for bin packing algorithms.

The algorithms REVISED FIRST FIT and MODIFIED FIRST FIT DECREASING are recent developments which we shall be discussing in detail shortly. Most of the other results in the table were already known by 1973 [69,70]. The algorithm BEST FIT (BF) is like FIRST FIT, except that  $p_i$  is placed in the bin into which it will fit with the smallest gap left over (with ties broken in favor of the lowest indexed bin) [72]. WORST FIT (WF) [69,70] places  $p_i$  in the non-empty bin with the biggest gap (ties broken in the same way), starting a new bin if this biggest gap is not big enough. ALMOST WORST FIT (AWF) [69,70] tries the second largest gap first, and then proceeds as does WORST FIT — surprisingly, this makes a difference. The analysis of NEXT FIT DECREASING was done by Baker and Coffman [7]. GROUP FIT GROUPED (GFG) [69,70] uses "implicit rounding" to discretize the ranges of item sizes and bin levels, thus avoiding the sorting implicit in the FFD algorithm which it mimics. It also attains a linear running time, while

paying only a partial penalty in worst case behavior. FIRST FIT GROUPED (FFG) [69,70] is a hybrid algorithm, included mainly because it yields a different value of  $R_A^\infty$ . The algorithm ITERATED LOWEST FIT DECREASING, which attains the same value, but more slowly, is due to Krause, Shen, and Schwetman [85], and will be discussed in more detail in the next section.

A variety of other results were also obtained during the early 1970's. The precise values of  $R_A^\infty(t)$  as a function of  $t$  were obtained for many of the algorithms [69,70,72]. Except for the algorithms WORST FIT and NEXT FIT, which yield the continuous function  $R_A^\infty(t) = 1+t(1-t)$ , these tend to be step functions determined by  $\lfloor 1/t \rfloor$ . In [69] the asymptotic worst case behavior of FIRST FIT was completely determined for the case when all item sizes lie in a specified interval, as a function of the interval. The algorithms NEXT- $k$  FIT,  $k \geq 1$ , which resemble NEXT FIT except that  $p_i$  is placed in a new bin only if it will not fit in any of the last  $k$  non-empty bins (NEXT-1 FIT is the same as NEXT FIT), were studied in [69,70]. These papers also analyzed what might be considered "non-deterministic" bin packing algorithms: ANY FIT (AF), which can place  $p_i$  anywhere, except that it can never put it in a new bin unless it won't fit in any of the already non-empty bins, and ALMOST ANY FIT (AAF), which in addition can never put  $p_i$  in a bin whose gap is larger than that of all other bins unless that is the only place it fits. The results for ALMOST ANY FIT are the same as those for FIRST, BEST, and ALMOST WORST FIT (all of which obey the AAF assumptions), while the results for ANY FIT are the same as those for WORST FIT, which essentially makes the worst choices allowed under the AF assumptions. ANY FIT DECREASING and all DECREASING algorithms obeying the ANY FIT ground rules seem to obey the same bounds as FFD, although the best that has been proved for any such algorithms (other than FFD and BFD) is that  $R_A^\infty \leq 5/4 = 1.25$  [69,70].

Another special class of algorithms that has received attention consists of the "on-line" algorithms. An on-line algorithm is one which, like NEXT FIT or FIRST FIT, assigns items to

bins in exactly the order they are given in the original list, without using any knowledge about subsequent items in the list. FIRST FIT DECREASING, for example, is not an on-line algorithm, since it first re-orders the list. On-line algorithms may be the only ones that can be used in certain situations, where the items to be packed are arriving in a sequence according to some physical process and must be assigned to a bin as soon as they arrive. Thus, although it is known that "off-line" algorithms such as FFD can do much better than FIRST FIT, it is of interest to determine the best worst-case performance that any on-line algorithm can have. On the basis of a clever analysis of the worst case examples for FIRST FIT, Yao [109] was able to devise a new algorithm, REVISED FIRST FIT (RFF), with  $R_{RFF}^{\infty} = 5/3 = 1.6666$ , which is to be compared with  $R_{FF}^{\infty} = 1.7$ . Even more significantly, he was able to show that for *any* on-line algorithm  $A$ , we must have  $R_A^{\infty} \geq 1.5$ . In subsequent work Brown [12] and Liang [91] independently extended Yao's lower bound results, improving the lower bound to 1.536. In addition Brown designed a further revision of FIRST FIT, whose asymptotic worst case ratio is better than 1.64 [14].

Galambos and Turan [46] quite recently considered the lower bound question for on-line algorithms when the list is assumed to be in non-increasing order. They showed that any such algorithm must have a worst-case bound that is at least  $10/9$ .

In [109] a slight improvement to FFD was also found. However, more significant improvements with an  $O(n \log n)$  running time not much worse than that of FFD were discovered by Friesen and Langston [45] and by Garey and Johnson [54]. The first of these employs a hybrid algorithm: Both FFD and an algorithm called BEST TWO FIT are run on the input; the output is taken as the better of the two packings produced. Friesen and Langston showed that, for any list, the *average* of the number of bins required by the two component algorithms can not exceed  $6/5 = 1.2$ , thus guaranteeing a weak upper bound on the minimum.

Garey and Johnson devised the MODIFIED FIRST FIT DECREASING algorithm for which the tight asymptotic bound,  $R_{MFFD}^{\infty} = 71/60 = 1.18333\dots$ , was proved. The algorithm is based on

a careful analysis of the  $11/9$  examples for FFD and what causes FFD to mispack them. It proceeds as follows: Partition the input list  $L$  into three sublists  $L_A = \{p_i: s(p_i) \in (\frac{C}{3}, C]\}$ ,  $L_D = \{p_i: s(p_i) \in (\frac{11C}{71}, \frac{C}{3}]\}$  and  $L_X = \{p_i: s(p_i) \in (0, \frac{11C}{71}]\}$ . The first step is to pack the sublist  $L_A$  using *FFD*. In the resulting packing, call a bin containing only a single item from  $L_A$  an " $A$ -bin." Then pack as much of  $L_D$  into  $A$ -bins as possible using the following loop:

1. Let bin  $B_j$  be the  $A$ -bin with the currently largest gap. If the two smallest unpacked items in  $L_D$  will not fit together in  $B_j$ , exit from the loop.
2. Let  $p_i$  be the smallest unpacked item in  $L_D$ , and place  $p_i$  in  $B_j$ .
3. Let  $p_k$  be the *largest* unpacked item in  $L_D$  that will now fit in  $B_j$ , and place  $p_k$  in  $B_j$ . Go to 1.

The assignment of items to bins is then completed by combining the unpacked portion of  $L_D$  with  $L_X$  and adding all these remaining items to the packing using *FFD*.

It should be noted that the proofs of the results for the more effective algorithms are characteristically long and intricate "weighting function" arguments. This proof technique originated with the analysis of FIRST-FIT [72], and plays a central role in the theory. A tutorial discussion of the use of weighting functions can be found in [22].

A number of the detailed proofs have been so long as to preclude their full publication in technical journals (e.g. there are results in [33,72,86] whose proofs span 100 pages). Baker [4] has illustrated, however, that significantly shorter proofs may be possible; Baker gives a proof of the basic  $11/9$  theorem which is about  $1/3$  the length of the original.

*Other Item Constraints* — As we have seen, when all piece sizes are known to be sufficiently small compared to  $C$ , the performance of approximation rules can improve substantially. Similar

improvements can be expected from other restrictions that occur frequently in practice. For example, suppose all item sizes are of the form  $C(\frac{1}{k})^j$ ,  $j \geq 0$ , for some fixed positive integer  $k$ . Then it is not difficult to show that  $FFD(L) = OPT(L)$ . As shown in [28] similar results hold for many of the approximation algorithms designed for the problems surveyed in the remaining sections of this paper. Note that power-of-two item sizes occur in important computer applications; by the nature of binary computers, if the sizes of records (files, pages, etc.) are constrained to be powers of two, algorithms for maintaining and allocating storage have much more efficient implementations.

As another illustration suppose the number of different item sizes is fixed, and therefore the number of bin types (i.e. the number of possible item-size configurations that fit into a bin) is finite. In this case, the work of Gilmore and Gomory [55,56] in the early 60's can be applied. (The importance of this work will emerge again shortly in our discussion of approximation schemes.) They were able to show that the linear programming relaxation of the problem, although still quite large (it has a variable for each bin type), can be solved using special techniques. An actual packing is then constructed by "rounding up" the solution values. In terms of worst case analysis, this algorithm will have  $R_A^\infty = 1$  for any fixed number of item sizes, since it can yield at most one excess bin for each possible bin type (a much larger, but still fixed number, independent of the number of items). We note in passing that when the number of item sizes is fixed, we actually can find *optimal* solutions in polynomial time, although the degree of the polynomial can be astronomical. Gilmore and Gomory's contribution is in obtaining *almost* optimal solutions with much less work.

*Approximation Schemes* — The prospects for improved approximation algorithms came to be much better understood as the result of two major results of the past three years. The first was the discovery by Fernandez de la Vega and Lueker [40] that for every  $\epsilon > 0$  there is a linear-time algorithm,  $A[\epsilon]$ , with  $R_{A[\epsilon]}^\infty \leq 1 + \epsilon$ . Algorithms of this type had long been known for problems

such as the knapsack problem, where performance is measured by absolute, rather than asymptotic worst-case ratios. A set of such algorithms  $\{A[\epsilon]: \epsilon > 0\}$  has been termed an *approximation scheme*. It was shown in [40] that techniques from the knapsack approximation scheme could in fact be used in designing bin-packing algorithms satisfying

$$A[\epsilon](L) \leq (1+\epsilon)OPT(L) + (1/\epsilon)^2,$$

and having a running time linear in the length of  $L$  for fixed  $\epsilon$ . The central idea in the proof is the reduction of the original bin-packing problem to one in which the number of possible item configurations in a bin is bounded. As in the Gilmore and Gomory [55] approach mentioned earlier the algorithm is formulated as a solution to a linear program.

Subsequently, Karmarkar and Karp [74] eliminated a shortcoming of the above result, viz. the fact that the running time of  $A[\epsilon]$  is exponential in  $(1/\epsilon)^2$ . Using the Fernandez de la Vega and Lueker results and an impressive array of techniques from mathematical programming and complexity theory they devised a "fully polynomial" approximation scheme, i.e. one for which the running time is a polynomial in both  $1/\epsilon$  and the length of  $L$ , and the additive constant is also a polynomial in  $1/\epsilon$ .

An interesting corollary to these results is that there exist polynomial time approximation algorithms with  $R_A^\infty = 1$ . One need only choose  $\epsilon$  as an appropriate function of the given instance. Unfortunately, the actual guarantee provided by these algorithms is not  $OPT(L) + K$  for some constant,  $K$ , but  $OPT(L) + f(OPT(L))$ , where  $f$  is a slowly growing function. The best such function comes from Karmarkar and Karp's analysis, and is  $O(\log^2(OPT(L)))$ .

At present the above results are mainly theoretical in their significance, because the coefficients hidden in the term "polynomial time" are too large for practical purposes. However, in principle at least, the search for better approximation algorithms can now take a different tack: Instead of trying to improve old bounds without great sacrifices in running time, we can try to improve on old running

times without great sacrifices in the bound.

*Average Case Analysis* — As might be expected from the greater difficulty in calculating probabilistic measures, the known results for the average case are less sharp and less general than those for the worst-case. However, as we shall see, the field is very active and a number of significant advances can be cited.

Before getting into the analytical approach let us consider what has been learned from Monte Carlo simulations. The most extensive experiments appear to be those in [69], and subsequently those in [96]. Our illustrations will be drawn from [69]. Since the results in [96] measure percentage of waste per bin rather than number of excess bins, they are not readily comparable with our worst case results. A summary of some of the results is shown in Table 2.

| Algorithm         | UNIFORM<br>(0,1.0) | UNIFORM<br>(0,0.5) | UNIFORM<br>(0,0.25) |
|-------------------|--------------------|--------------------|---------------------|
| NEXT FIT          | 31.1 [100.]        | 18.8 [100.]        | 7.4 [50.0]          |
| NEXT-2 FIT        | 21.9 [85.0]        | 8.5 [50.0]         | 2.2 [25.0]          |
| ALMOST WORST FIT  | 10.4 [70.0]        | 4.8 [50.0]         | 1.4 [25.0]          |
| FIRST FIT         | 7.0 [70.0]         | 2.2 [50.0]         | 0.6 [25.0]          |
| BEST FIT          | 5.6 [70.0]         | 2.2 [50.0]         | 0.5 [25.0]          |
| GROUP FIT GROUPED | 2.1 [50.0]         | 0.4 [33.3]         | 0.3 [20.0]          |
| AWF DECREASING    | 2.0 [22.2]         | 0.2 [18.3]         | 0.2 [15.0]          |
| FF DECREASING     | 1.9 [22.2]         | 0.1 [18.3]         | 0.2 [15.0]          |
| BF DECREASING     | 1.9 [22.2]         | 0.1 [18.3]         | 0.2 [15.0]          |

**Table 2.** Percentages of excess bins required on the average in bin-packings of 25 200-item lists with item sizes uniformly distributed within the stated ranges. [Percentage of excess in worst examples known are given in brackets.]

We should note that the ratios given are not strictly speaking averages of  $R_A(L)$ , since the value of  $OPT(L)$  could not be determined (its computation being an *NP*-complete problem). Instead these values are for the ratio of  $A(L)$  to the sum of the item sizes. As we shall see later, however, there is strong evidence to support the claim that this approximation loses very little when  $L$  is large.

The interesting fact from these simulations is that the average behavior, although much better than the worst case behavior, still ranks the algorithms in the same relative order. Results for an approximation to a normal distribution, and for a distribution obtained by partitioning a set of items of size  $C$  into a random number of items, are slightly worse, but reflect the same trends [37]. It should not be expected, however, that average case ranking will always reflect worst case ranking. In particular, certain of the new algorithms specifically designed for improved worst case behavior (although possibly not MFFD or the hybrid algorithm in [45]) may be comparatively bad on the average.

Because of the restriction to 200-item list the results in Table 2 are not indicative of asymptotic behavior. Extensive simulation studies of asymptotic performance are currently in progress by Jon Bentley and Catherine McGeoch at Bell Laboratories. An interesting sample of their early results is that the expected performance of  $FF$  is  $n/2 + H(n^{0.8})$  for  $C$  normalized to 1 and for  $n$  items with sizes uniformly distributed over  $[0,1]$ .

The first mathematical results appeared in an approximate analysis by Shapiro [102]. The approximation was based on the exponential distribution and estimated the expected value, given  $NF(L)$ , of  $OPT(L)$ . He concluded that as  $NF(L)$  approaches infinity,  $R_{NF}(L)$  approaches 1 plus the average item size, when that average is  $C/5$  or less.

The first exact results were obtained by Coffman, So, Hofri and Yao [32] from an analysis of a Markov process defined on the bin levels. For item sizes uniformly distributed between 0 and  $C$ , it was shown that  $E(NF(L))$  is bounded by  $(4/3)E(OPT(L))+4$ . It was also shown that convergence to the stationary NEXT FIT bin-level distribution was exponentially fast. Hofri [68] and Ong, Magazine and Wee [97] have recently strengthened these results by working out the second moment of  $NF(L)$ , and Hofri has derived an approximate probability generating function valid for general item size distributions.

Karmarkar [73] subsequently extended the  $NF$  results to cover uniform distributions over any interval  $(0, tC]$ . Closed forms were obtained for  $\frac{1}{2} \leq t \leq 1$ , and gave results within about .5% of the empirical results in [96]. It is perhaps natural to expect that the stationary, expected bin-level would increase monotonically with decreases in the maximum piece size,  $t$ . However, as observed in the data of [96] and confirmed analytically in [73], this is not the case for NEXT FIT. In fact, packing efficiency was found to be least at  $t = 0.841$ , which agrees with empirically observed behavior.

Results on algorithms other than NEXT FIT have also been obtained for uniformly distributed item sizes. In [42], Frederickson gives a rigorous proof of the rather intuitive fact that, for this particular distribution, the ratio  $E(FFD(L))/E(OPT(L))$  approaches 1 as  $n \rightarrow \infty$ , where  $n$  is the number of items. (Note that this ratio is not the same as  $E(FFD(L)/OPT(L))$ .) Frederickson achieves this result by analyzing a different algorithm which produces packings with a much simpler structure. Basically, working inward from both ends of an ordered list of the pieces, the algorithm attempts to pair large pieces with small ones. The analysis shows that the algorithm requires  $n/2 + O(n^{2/3})$  bins, on the average, to pack a list of  $n$  pieces with  $C = 1$ .

Early results on the asymptotic properties of optimal packings were obtained by Loulou [93]. He proved that if the  $n$  item sizes are independent and identical random variables with the density function  $f$ , then the limit in probability,  $\bar{R}$ , of the ratio of  $E(OPT(L))$  to the expected total of the item sizes divided by  $C$  is 1 as  $n \rightarrow \infty$ , if  $f$  is symmetric or if  $f$  is positive and decreasing over  $[0, C]$ . Karmarkar [65] recently showed that this asymptotic result also holds if  $f$  is decreasing over  $[0, tC]$ ,  $0 < t < 1$ , and 0 elsewhere.

Results of this type have important consequences for the analysis of approximation algorithms. They suggest that for reasonably large lists, the approximation of  $OPT(L)$  by the sum of the item sizes divided by  $C$  is indeed a very good one for large classes of item size distributions.

Lueker [95] studied the same question specialized as follows: If  $f$  is uniform over  $[a, b]$ ,  $b > a$ , for what values of  $a$  and  $b$  does  $\bar{R} = 1$ ? For  $a = 0$  or  $a+b = 1$  it was known that  $\bar{R} = 1$ , and for points in the region  $b > 1-a$  it was known that  $\bar{R} > 1$ . Lueker identified a substantial subset of the region  $0 < a < b < 1-a$  where  $\bar{R} > 1$ , and cited simulation results which suggest that  $\bar{R} = 1$  elsewhere in this region.

Knödel [82] and Lueker [94] have extended Frederickson's results for the uniform distribution to other approximation algorithms. With a slight modification of Frederickson's algorithm ( $C=1$ ) Lueker showed that the expected number of bins used can be tightened to  $n/2 + O(n^{1/2})$ , a result that also applies to Knödel's algorithm. Within a multiplicative constant this is the best possible performance, since Lueker also showed that an *optimal* algorithm must use  $n/2 + \Theta(n^{1/2})$  bins on the average. In [76] Karmarkar, Karp, Lueker and Murgolo generalized these results to any probability density symmetric about  $C/2$  or positive and decreasing over  $[0, C]$ , thus strengthening Loulou's earlier result.

Hoffmann [66] and more recently Lee and Lee [90] have considered more deeply the expected performance of *on-line* algorithms. They have designed algorithms based on "reservation" techniques whereby bins are dedicated to particular configurations of items. In [66] these configurations are based on those of Frederickson's pairing algorithm. Hoffmann has shown that his on-line algorithm retains asymptotic optimality (in the earlier expected value sense), but at the expense of a poor worst-case performance.

Lee and Lee develop an algorithm with a good worst-case performance and with a good, though not asymptotically optimal, expected performance that is relatively easy to analyze. Their on-line HARMONIC algorithm,  $H$ , packs items so that the sizes of all items in any given bin are in the same interval of the set  $\{(\frac{C}{2}, C], (\frac{C}{3}, \frac{C}{2}], \dots, (0, \frac{C}{M}]\}$ , where  $M \geq 2$  is a parameter of the algorithm. They show that  $R_H^\infty(M) \leq 1.692$  for all  $M \geq 12$ , and that

$\lim_{M \rightarrow \infty} R_H^\infty(M) = 1.691\dots$ , thus improving on both NEXT FIT and FIRST FIT. The latter bound,

$$1.691\dots = 1 + \frac{1}{2} + \frac{1}{2 \times 3} + \frac{1}{6 \times 7} + \frac{1}{42 \times 43} + \dots,$$

is the same as the one given for NF DECREASING in Table 1. (A worst-case analysis of NF DECREASING [7] also entails the analysis of HARMONIC packings.)

The average-case performance of HARMONIC is analyzed for several item size distributions. For the uniform distribution over  $[0, C]$  they show that  $E(H(L))/E(OPT(L))$  is asymptotically no greater than 1.29 for all  $M > 12$ , which is to be compared with the corresponding  $4/3$  result for NEXT FIT.

### 3. Bin-Packing Variants

In this section we survey results for variants on the classical one-dimensional bin packing problem in which the goal is still to minimize the number of bins used.

*Constraints on the Number per Bin* — This modification was considered by Krause, Shen, and Schwetman [85] as a model for multiprocessor scheduling under a single resource constraint when the number  $k$  of processors is fixed. In this case the items represent tasks to be executed, with the size of an item being the amount of the resource it requires (out of a total quantity of  $C$ ). If we assume that all tasks have the same unit-length execution time, then a schedule corresponds to an assignment of tasks to integral starting times, such that at no time are there more than  $k$  tasks being executed or is there more than  $C$  of the resource being used. The objective is to minimize the latest starting time. This corresponds to bin packing where the bins represent starting times and can contain at most  $k$  items.

Krause et al. analyze three algorithms for this problem. The first two are just FIRST FIT and

FIRST FIT DECREASING, suitably modified to account for the bound on the number of items per bin. The results are simply stated:

$$\frac{27}{10} - \left\lceil \frac{37}{10k} \right\rceil \leq R_{FF}^{\infty} \leq \frac{27}{10} - \frac{24}{10k}; \quad R_{FFD}^{\infty} = 2 - \frac{2}{k}$$

Note that as  $k \rightarrow \infty$ , these bounds remain substantially worse than the corresponding bounds when the number of items per bin is not restricted (27/10 versus 17/10 and 2 versus 11/9). Thus the very existence of a limit, and not just its size, can have a substantial effect on the worst case behavior of the algorithms.

The third algorithm studied was alluded to in the previous section. ITERATED LOWEST FIT DECREASING uses a technique we shall be meeting again in the next section, so we shall describe it in detail. We first put the items in non-decreasing order by size, as we do for *FFD*. We then pick some obvious lower bound  $q$  on  $OPT(L)$  and imagine we have  $q$  empty bins,  $B_1, B_2, \dots, B_q$ . Place  $p_1$  in  $B_1$  and proceed through the list of items, packing  $p_i$  in a bin whose current contents has minimum total size (breaking ties by bin index, when necessary). If we ever reach a point where  $p_i$  does not fit in *any* of the  $q$  bins (either because the capacity  $C$  or the limit  $m$  is exceeded), we halt the iteration, increase  $q$  by 1, and start over. Eventually we will succeed in generating a packing for some value of  $q$ , and this will be the output.

The running time of *ILFD* is  $O(n^2 \log n)$ , but this can be improved to  $O(n \log^2 n)$  by using binary search on  $q$ . The performance bound proved for *ILFD* is  $R_{ILFD}^{\infty} \leq 2$ , which makes *ILFD* competitive with *FFD*. It is conjectured that the actual value of  $R_{ILFD}^{\infty}$  is closer to the 4/3 value we cited in the last section for the case when there is no limit on the number of items per bin.

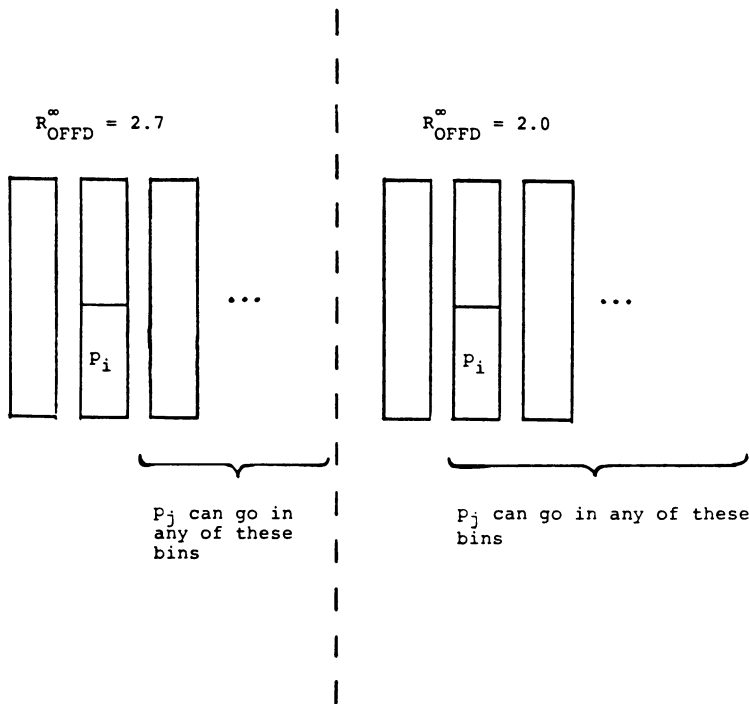
*Partial Orders on L* – Partial orders,  $\leq$ , on the set  $L$  of items arise in two potential applications of bin-packing. One is again related to multiprocessor scheduling, and was studied by Garey, Graham, Johnson and Yao in [50]. Suppose we have a set of unit-length tasks  $p_1, \dots, p_n$  with resource

requirements subject to an over-all bound of  $C$ , as above, but with no limit on the number of processors. In this case a partial order  $\leq$  is interpreted as follows:  $p_i \leq p_j$  means that  $p_i$  must be executed *before*  $p_j$ , i.e., must be assigned to a bin with lower index than that to which  $p_j$  is assigned.

The other application in which partial orders arise is in "assembly line balancing," and is studied by Wee and Magazine [107]. Here the items represent tasks to be performed on a single product as it moves along an assembly line. Each is performed at one of a sequence of workstations  $B_1, B_2$ , etc., and the item sizes correspond to the times required to execute the tasks. The assembly line advances in discrete steps, stopping for a period of time  $C$  at each workstation. Thus a set of tasks can be assigned to a work station (bin) if their total time (size) does not exceed  $C$ . The goal is to minimize the number of workstations required. In this case a partial order  $\leq$  has the following interpretation:  $p_i \leq p_j$  means that in any assignment of tasks to workstations (bins),  $p_i$  must be performed before  $p_j$  (but they could be performed at the same workstation, merely by doing  $p_i$  before  $p_j$  within the total time  $C$  allowed, so this time  $p_i$  can go either in an earlier bin or the *same* bin as  $p_j$ ).

Note that these two applications yield different interpretations of the partial order constraint within the bin packing context. See Figure 4. Although this difference might appear to be slight, its consequences, as shown in the figure, are nontrivial. The algorithm referred to there, ORDERED FIRST FIT DECREASING, is the best algorithm known for either version of the problem, but yields quite different guarantees. It is quite simple to describe. First, we order the items by non-increasing size, as with *FFD*. We then pack bins, rather than items, in sequence. Bin  $B_i$  is packed as follows: Place the largest unpacked item into  $B_i$  that the partial order will allow. Repeat until no more items can legally be packed into  $B_i$ .

Note that this algorithm can be applied to either version of the problem, so long as the partial



**Figure 4.** Two interpretations of  $P_i \lesssim P_j$  and their consequences.

order is interpreted appropriately. Note also that, in the absence of a partial order, this algorithm generates the same packing as FIRST FIT DECREASING and hence has an asymptotic worst-case ratio of  $11/9$ .

**Clustered Items** — Here, the basic idea is that only "closely related" items may go in a bin together. The one example we cite is from a paper by Chandra, Hirschberg, and Wong [18], although other potential applications of this type might come to mind. Here the items are thought of as having geographical locations. Putting them in the same bin corresponds to assigning them to a common facility (computing service, telephone switching center, etc.), where each such facility is assumed to have a standard capacity  $C$ . We desire that the items which are served by a common facility be in

close proximity to each other, and this restricts the types of packings that we allow. The main results in [18] concern the case when the contents of a bin must all reside within the same unit square, although other figures, such as unit circles, are also considered. For the unit square case, a geometric algorithm is proposed and shown to have  $R_A^\infty$  lying between 3.75 and 3.8.

*Dynamic Bin-Packing* — This variant was proposed originally as a model of certain problems in computer storage allocation; however, storage applications in a general industrial setting are easily envisioned. In the computer application the bins correspond to storage units such as disk cylinders, and the items correspond to records which must be stored for certain specified periods of time. Associated with an item is thus not only a size  $s(p_i)$ , but also a beginning time  $b(p_i)$  and an ending time  $e(p_i)$ . A packing is an assignment of items to bins such that at any time  $t$  and for any bin  $B$ , the items assigned to that bin which have begun by time  $t$  and not yet ended by that time have total size no greater than the capacity  $C$ .

The research reported by ourselves in [27] concentrates on "on-line" algorithms, where in this case an on-line algorithm packs items in the order in which they begin, and may not use information about items which are to begin later, or the ending times for items which are currently in the packing (this lack of information mirrors the predicament often faced by actual computer storage allocators). It is assumed that once an item is assigned to a bin it cannot be moved to another bin.

The algorithm FIRST FIT can be readily adapted to this situation, but the dynamic nature of the environment significantly impairs its performance. For the case when no item size exceeds  $C/2$ , we have  $R_{FF}^\infty(1/2) = 1.5$  in the classical case, but in the dynamic case it is shown in [27] that *any* on-line algorithm must obey  $R_A^\infty(1/2) \geq 5/3 = 1.666\dots$ . For FIRST FIT it was proved that  $R_{FF}^\infty(1/2)$  lies somewhere between 1.75 and 1.78. The case when items larger than  $C/2$  are allowed is even more difficult to analyze (as seems usually to be the case with bin packing), but is clearly much worse. Here it is known that  $R_{FF}^\infty$  lies somewhere between 2.75 and 2.89, and *any* on-

line algorithm must obey  $R_A^\infty \geq 2.5$ .

Studies of dynamic packings are at the interface between bin-packing and dynamic storage allocation. The latter class of problems is distinguished by the assumption that items, once packed, can not be moved at all prior to their departure. (In dynamic bin packing the allowed movement of items *within* bins was implicit; only the movement of an item from one bin to another was disallowed.) Under this added assumption the fragmentation (alternating holes and occupied regions) that develops as items come and go can create far more wasted space; the space wasted by the partitioning of storage into "bins" is usually minor by comparison. The standard model considers only a single bin whose capacity is to be determined under a given algorithm and the assumptions: the total size of items present at any time never exceeds  $m$  and the maximum item size is  $j$ . (Note that  $m$  is our usual lower bound on the capacity required by an optimal algorithm.)

FIRST FIT and BEST FIT are the principal approximation algorithms that have been studied. With FIRST FIT an arriving item is stored at the beginning of the first sufficiently large hole encountered in a scan of the bin. BEST FIT is defined similarly, where the hole is a smallest one exceeding the item size. Let  $C_A(j, m)$  denote the capacity needed in the worst-case under algorithm  $A$ , and in the spirit of our other asymptotic bounds let  $C_A^\infty(j) = \lim_{m \rightarrow \infty} C_A(j, m)/m$ . Robson [99] has shown that  $\frac{1}{2} \log_2 j \leq C_{FF}^\infty(j) \leq \log_2 j$  and  $C_{BF}^\infty(j) = \Theta(j)$ . The bound for FIRST FIT is a best possible one in the sense that an optimal algorithm must have a  $\Theta(\log j)$  worst case if, like  $FF$  and  $BF$ , it must allocate storage to each item at its time of arrival, and if no information is available on items that have not yet arrived. This last result of Robson [98] is in fact the classical one of dynamic storage allocation, and generalizes earlier results of Graham [61].

There are many other interesting results for this problem, particularly those specializing item sizes to powers of two. Knuth [83] covers the elements of the subject and a recent survey appears in [23].

#### 4. Multiprocessor Scheduling

We continue here with the bin-design problem defined in the Introduction: With the number of bins fixed, find the smallest, common bin capacity  $C$  sufficient to pack  $L$ .

The initial work on approximation algorithms for this problem appears in [60,62]. We can define worst case ratios as we did before, noting that in this case the asymptotic and absolute worst case ratios will coincide for reasonable algorithms — any worst case example can be converted to one with an arbitrarily large value for  $C$  merely by scaling up all the sizes by an appropriate multiplicative factor. (In those applications in which there is a fixed upper bound on the possible item sizes, asymptotic worst case bounds would make sense, but due to the nature of the problem we would tend to get  $R_A^\infty = 1$  for most algorithms, e.g., see [80]). Thus we shall express results for this problem in terms of the absolute ratio  $R_A$ . Graham examined two basic algorithms. LOWEST FIT assigns the items to bins in order, placing  $p_i$  in a bin with current contents of minimum total size (ties broken by bin index when necessary). LOWEST FIT DECREASING first sorts the items so that they are in non-increasing order by size and then applies LOWEST FIT. Fixing  $m$ , the number of bins, Graham was able to prove that  $R_{LF} = 2 - (1/m)$  [60] and that  $R_{LFD} = (4/3) - (1/3m)$  [62]. In analogy with the results for  $R_A^\infty(t)$  in the classical problem, Coffman and Sethi [35] showed that the  $LFD$  bound improves to  $R_{LFD}(k) \leq \frac{k+1}{k} - \frac{1}{km}$  when it is known that there are at least  $k \geq 3$  items per bin. In [88] Langston demonstrated how the bad examples of  $LFD$  could be effectively avoided. The resulting algorithm,  $LFD^*$ , uses an iterative technique and has a worst-case bound,  $R_{LFD^*} \leq \frac{5}{4} + \frac{1}{12}(2^{-k})$ , where  $k$  is the number of iterations chosen.

Sahni [100] developed an approximation scheme for this problem several years ago. In particular, he showed that for any fixed value of  $m$  and any  $\epsilon > 0$  there exists a polynomial time

algorithm with  $R_A = 1 + \epsilon$ . As in the corresponding studies for the bin-packing problem discussed earlier, the result is primarily of theoretical interest; such algorithms are exponential in  $m$  (and polynomial in  $1/\epsilon$ ) and therefore unattractive for  $m > 3$  or very small  $\epsilon$ .

A more practical algorithm which is polynomial in  $m$  and still improves upon *LFD* was presented by ourselves in [26]. The algorithm works on an iterative principle much like that of *ILFD*. Called MULTIFIT DECREASING, the algorithm works by guessing a capacity  $C$  and then applying *FFD* to the list. The next guess is either larger or smaller, depending on whether *FFD* used more than  $m$  bins of that capacity to pack the list or not. By using an appropriate binary search strategy and limiting the number of iterations performed to some small number  $k$ , we obtain an algorithm, denoted  $MF(k)$ , for which we proved the bound  $R_{MF[k]} \leq 1.220 + (1/2)^k$ . Friesen [44] subsequently established the improved upper bound,  $6/5 + (1/2)^k$ , which again is independent of  $m$ .  $MF(k)$  improves upon *LFD* for all  $m > 2$  when  $k \geq 5$ , at the cost of only a small increase in running time. The worst behavior known for this algorithm is shown in examples constructed by Friesen [44], which imply that  $R_{MF[k]} \geq 13/11 = 1.18181\dots$  for  $m \geq 13$ . (Better upper bounds are known for the cases when  $m \leq 7$  [26]).

We remark in passing that the results for MULTIFIT DECREASING are proved by considering the following bin packing variant: Suppose we are given two sets of bins, one with bins all of capacity  $\alpha$ , the other with bins all of capacity  $\beta$ . What is the asymptotic worst case ratio of the number of  $\beta$ -capacity bins used by *FF* (or *FFD*) to the minimum number of  $\alpha$ -capacity bins needed? This question, first raised in [49], is investigated in detail in Friesen's thesis [44] as well as in [26], and is used in [49] for proposing a conjectural explanation of the mysterious fraction  $17/10$  in the original theorem about  $R_{FF}^\infty$  (a conjecture that, unfortunately, is only partially true [103]).

An algorithm structurally different from any considered so far was introduced by Finn and Horowitz [41] and later improved by Langston [89]. It is based on finding approximate solutions to

the following closely related partitioning problem: Partition  $L$  into  $m$  blocks (bins)  $B_1, \dots, B_m$  so as to minimize the difference in the maximum and minimum bin levels, viz.

$$D(B_1, \dots, B_m) = \max_i \{\ell(B_i)\} - \min_i \{\ell(B_i)\},$$

where  $\ell(B_i) = \sum_{p \in B_i} s(p)$ . As is to be expected, good approximate solutions for this problem are good approximate solutions for the problem of minimizing bin capacity. (The two problems are equivalent, of course, for  $m = 2$ .) The basic idea of the algorithm is iteratively to exchange items in the two bins having the maximum and minimum levels until  $D$  can no longer be reduced in this way.

As shown in [41] linear time algorithms can produce packings comparing favorably with those of MULTIFIT DECREASING, especially for large lists. Quite recently, however, Karmarkar and Karp [75] obtained even stronger results. Again concentrating on better performance for a large number,  $n$ , of items, they devised algorithms based on an operation called set differencing. These algorithms are best illustrated for the case  $m = 2$ .

The differencing operation consists of selecting a pair of item sizes  $s(p)$  and  $s(p')$  from  $L$  and then restricting the solution to partitions in which  $p$  and  $p'$  appear in different bins. The new, smaller problem is then equivalent to partitioning

$$L' = L - \{s(p), s(p')\} \cup \{|s(p) - s(p')|\}.$$

Consider for example, the algorithm: *While*  $|L| > 1$ , *iteratively select the largest two elements*  $s$  *and*  $s'$  *of*  $L$  *and perform the operation*  $L \leftarrow L - \{s, s'\} \cup \{|s - s'|\}$ . The last number, when  $|L| = 1$ , determines  $D(B_1, B_2)$ ; the corresponding partition is trivial to construct by backtracking through the sequence of differencing operations. Figure 5 shows an example in the form of a tree. Other set-differencing algorithms can be obtained simply by altering the order in which elements are selected for differencing.

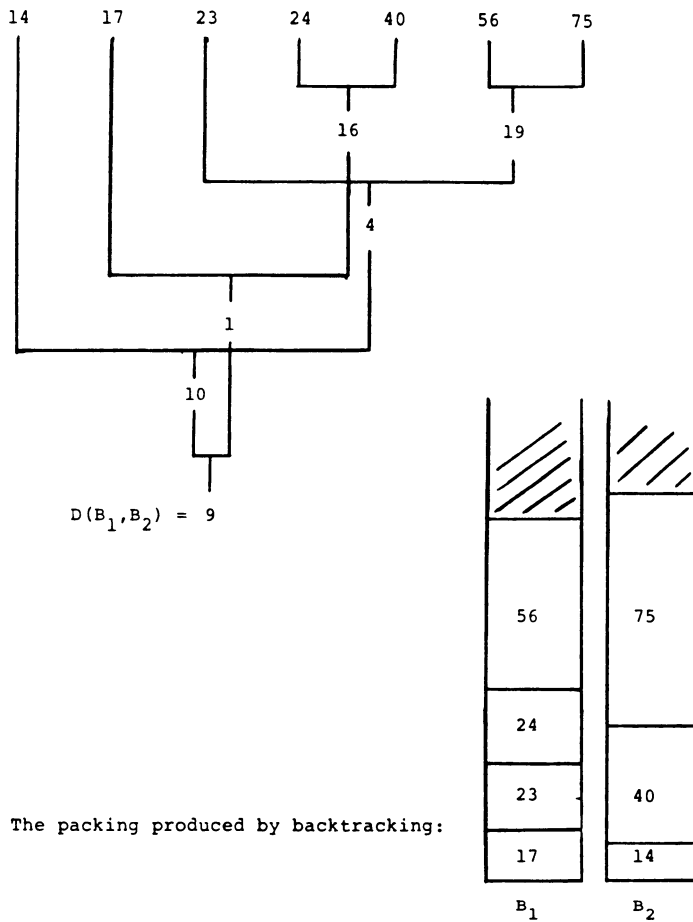


Figure 5. An Illustration of Set-Differencing.

The methods for  $m = 2$  are extended in a natural way to classes of algorithms for arbitrary  $m > 2$ . For a particular linear-time algorithm Karmarkar and Karp show that, except in pathological cases, partitions are produced with  $D = O(n^{-\log n})$ . This result is to be contrasted with the corresponding  $O(n^{-1})$  result that applies to *LFD* and *MULTIFIT*. To show that the pathological cases are extremely rare they use a simple probability model to verify that as  $n \rightarrow \infty$  the algorithm performs as claimed with probability 1. We shall return briefly to this analysis later on.

*Average-Case Analysis* — The first such analysis of a specific approximation algorithm was by Coffman, Frederickson and Lueker [24]. They analyzed the *LFD* rule for  $m = 2$  and showed that for item sizes uniformly distributed over  $[0,1]$

$$\frac{n}{4} + \frac{1}{4(n+1)} \leq E[LFD(L)] \leq \frac{n}{4} + \frac{e}{2(n+1)}.$$

Note that  $n/4$  is a lower bound on  $E[OPT(L)]$ . Shortly thereafter they proved that  $\frac{n}{2m} + O(\frac{m}{n})$  was the corresponding upper bound for the general case,  $m \geq 2$  [25].

More recently Bruno and Downey [16] analyzed the *LF* rule assuming independent, uniformly distributed item sizes. Their main result is an estimate of the tail probability  $P\{\frac{LF(L)}{OPT(L)} > x\}$  by which they were able to demonstrate numerically a fast convergence in probability of  $LF(L)$  to  $OPT(L)$  as  $n \rightarrow \infty$ . Using different techniques, Coffman and Gilbert [30] subsequently improved the bound on the tail probability and extended the analysis to exponentially distributed item sizes. Exponential convergence was demonstrated analytically for the latter distribution and an even faster convergence was found for the uniform case. Bounds on the expected values of the performance ratios were also derived for both cases. They are

$$\begin{aligned} \text{uniform: } E\left[\frac{LF(L)}{OPT(L)}\right] &\leq 1 + \frac{2(m-1)}{n-2}; \quad n > 2 \\ \text{exponential: } E\left[\frac{LF(L)}{OPT(L)}\right] &\leq 1 + \frac{(m-1)H_{m-1}}{n-m}, \quad n > m, \end{aligned}$$

where  $H_i$  is the  $i$ th harmonic number.

Asymptotic results of a more general nature were proved by Dempster et al. [36]. They showed that for any approximation algorithm,  $A$ , in a broad class (including all those considered in this survey), the ratio of  $A(L)$  to  $OPT(L)$  converges in probability to 1 as  $n \rightarrow \infty$ , if the item sizes are independent and identically distributed random variables with finite variance. Loulou [92]

subsequently proved a stronger result for the  $LF$  and  $LFD$  rules, viz. the absolute errors  $LF(L) - OPT(L)$  and  $LFD(L) - OPT(L)$  converge in probability to finite random variables.

Quite recently Frenk and Rinnooy Kan [43] dealt successfully with a conjecture of Loulou; they proved that  $LFD(L) - OPT(L)$  converges in probability to 0 as  $n \rightarrow \infty$  if the item size distribution has a finite mean and a density  $f$  satisfying  $f(0) > 0$ . They also show that if the distribution is uniform or exponential, the rate of convergence is  $O\left(\frac{\log n}{n}\right)$ .

What appear to be the strongest results of this type currently known were obtained recently by Karmarkar and Karp [75] for one of the set-differencing algorithms discussed earlier in this section, where the objective function is again the difference  $D$  between the largest and smallest bin levels. For the particular algorithm used in [75] let  $D^*(L)$  denote the output for a given  $m$ , and define

$$D_{m,n}^* = \sup_{\{L: |L| \leq n\}} D^*(L) .$$

They showed that there is a positive constant  $\alpha$  such that

$$D_{m,n}^* = e^{-\alpha(\ell n n)^2/m}$$

with probability 1 as  $n \rightarrow \infty$  for any item size distribution satisfying a mild smoothness condition.

A similar result is conjectured for the simple largest-pair-first algorithm illustrated in Fig. 5. However, a rather more elaborate, but still linear-time algorithm had to be adopted in order to avoid the usual difficulties in dealing with order statistics, as well as certain other problems.

To help motivate the above result let us assume  $m = 2$ , where the simpler form  $D_{2,n}^* \leq O(n^{-\alpha \log n})$  applies. Consider the following set-differencing algorithm which assumes that  $n = 2^k$ ,  $k \geq 1$ . Pair the largest two numbers in  $L$ , the next largest two, and so on. Differencing each pair we establish a reduced problem containing  $n/2 = 2^{k-1}$  differences. The desired partition is obtained from this process repeated  $k$  times. Under general assumptions it is reasonable to

expect that at the end of each of the  $k$  stages the order of magnitude of the numbers is reduced approximately by a factor of  $n$ . Accordingly, we expect a final partition such that  $D(L) = O(n^{-k})$  or  $D(L) = O(n^{-\log n})$ .

A result in a similar vein was recently proved by Karmarkar, Karp, Lueker and Odlyzko [77] for a problem posed by Michael Steele [105]. The problem was to find the rate of convergence of  $E[D(L)]$  as  $n \rightarrow \infty$  assuming an optimal algorithm,  $m = 2$  and item sizes uniformly distributed over  $[0,1]$ . Using a technique called the second-moment method, they showed that  $P[D(L) > n^2 2^{-n}] = O(n^{-1})$ .

## 5. Other Performance Criteria

*Sums of Squares of Bin Levels* — In parallel efforts Cody and Coffman [21] and Chandra and Wong [17] studied bin packing problems arising in the allocation of records on computer auxiliary storage devices. The basic probability models are patterned after those analyzed by Knuth [83] in connection with similar storage assignment problems. In bin packing terms the "sizes" of items are the access frequencies of the corresponding records.

The problem in [21] models paging drums where a given set of pages is to be partitioned among the  $m$  sectors (bins) of the drum so as to minimize average access time. This quantity is calculated to be  $\frac{m-2}{2} + \frac{m}{2} \sum_{i=1}^m \ell^2(B_i)$ , where  $\ell(B_i)$  is the  $i$ th bin level, i.e. the sum of the access frequencies of the pages in the  $i$ th sector. The minimization of this sum can be accomplished roughly by making the bin-levels all as close to each other as possible. In [21] the *LFD* rule is applied to this problem and the following result proved:  $R_{LFD} \leq 1 + \frac{1}{16(m-1)}$ .

The problem in [17] models arm contention in disk-pack computer storage. In this case, the object is to minimize the contention that occurs whenever two items from the same bin are

requested at the same time. Contention is measured here by the simpler quantity,  $\sum_{i=1}^m \ell^2(B_i)$ . The algorithm *LFD* is analyzed in this context too, and it is shown that for this problem  $\frac{37}{36} \leq R_{LFD} \leq \frac{25}{24}$ . In [38], Easton and Wong consider the variant in which no bin can contain more than  $k$  items. They analyze an appropriately modified version of *LFD* and showed that  $R_{LFD} \leq 4/3$ .

Wong and Yao [108] consider yet another variant based on minimizing access time [110]. Suppose we wish to *maximize*, rather than minimize, the sum of squares in the above case where  $k$  items per bin are allowed. This might be considered as a bin packing problem where all items have both a size  $s(p) = 1$  and an arbitrary weight  $w(p)$ , the bin capacity is  $k$ , and the goal is to pack the items into  $m$  bins so as to maximize the sum of the squares of the total weight in each bin. As observed in [110], this is of course an easy matter: merely put the  $k$  largest items in the first bin, the next  $k$  largest in the second bin, etc. Wong and Yao consider the generalization where the sizes as well as the weights are arbitrary.

In order that results for this maximization problem can be compared directly to those for the minimization problems we have been studying so far, we shall define  $R_A(L)$  to be  $OPT(L)/A(L)$  for any approximation algorithm  $A$  (this is the inverse of our definition for minimization problems).  $R_A$  and  $R_A^\infty$  are then defined as before and lie in the range  $[1, \infty)$ . Wong and Yao propose a heuristic based on ordering the items by non-decreasing density (weight divided by size) and then applying NEXT FIT. They show that this heuristic satisfies  $R_A \leq 2$ .

*Maximizing the Number of Items Packed* — We consider again a maximization problem that fixes the number of bins and the bin capacity. This time the goal is to pack as many of the items in  $L$  as possible into the bins. Coffman, Leung, and Ting [34] consider the algorithm FIRST FIT INCREASING, which first sorts the items into non-decreasing order by size, and then applies *FF*

until an item is reached which will not fit in any of the bins (which implies that none of the remaining items will fit either). They show that  $R_{FFI} = 4/3$ . In [33], Coffman and Leung consider an algorithm that, like *ILFD* and *MFD*, involves iteration. Their algorithm, denoted *FFD\**, works as follows: First sort the items in non-increasing order by size, and then apply *FF*. If some item fails to fit, stop, delete the first (largest) item in the list, and reapply *FF* to the shorter list. Repeat this until a list is obtained that *FF* does pack into the  $m$  bins. Coffman and Leung show that *FFD\** will always pack at least as many items as *FFI*, and indeed obeys the better bounds  $8/7 \leq R_{FFD^*}^\infty \leq 7/6$ , making the added complexity of *FFD\** over *FFI* worth the effort. Langston [87] has recently analyzed these heuristics for the more general model in which bin sizes may vary. By arranging bin sizes in non-decreasing order, he proves that  $R_{FFI}^\infty = 2$  and  $11/8 \leq R_{FFD^*}^\infty \leq 3/2$ .

*Maximizing the Number of Bins above a Given Level* — Suppose a threshold  $T > 0$  is given. Assmann, Johnson, Kleitman and Leung [3] studied the problem of finding a packing of  $L$  into a maximum number of bins such that each bin has a level not less than  $T$ . Similar to the sum of squares problem, for a good packing, the goal is roughly to pack every bin to a level as close as possible to, but not less than  $T$ .

Two approximation algorithms were examined for this problem. The first that we shall describe begins by producing a standard *FFD* packing of  $L$  for some given capacity  $C > T$ . The second stage iteratively takes items from the last non-empty bin and places them in the currently lowest indexed bin having a level less than  $T$ ; at the end of this stage some, possibly empty subset of highest indexed bins in the *FFD* packing will have been emptied in order to bring the levels of lower indexed bins up to at least  $T$ . The performance guarantee for this rule, called *FFD*[ $C$ ], is an interesting function of the value of  $C$  chosen for the initial *FFD* stage. Using our inverted ratio  $OPT(L)/A(L)$  as before, it is shown in [3] that  $R_{FFD(C)}^\infty \geq 3/2$  for all  $C \geq T$  and

$\lim_{C \rightarrow T} R_{FFD[C]}^\infty = 2$  and  $\lim_{C \rightarrow 2T} R_{FFD[C]}^\infty = 2$ , whereas  $R_{FFD[C]}^\infty = 3/2$  if  $\frac{4}{3}T \leq C < \frac{3}{2}T$ . In other words, for best worst-case performance we should choose  $C$  in  $[\frac{4}{3}T, \frac{3}{2}T)$ .

The second algorithm investigated was simply ITERATED LOWEST FIT DECREASING adapted to this problem. A value is guessed for the number,  $m$ , of bins in which to apply *LFD*. If each bin in the *LFD* packing has a level at least  $T$ , the algorithm halts. Otherwise, a smaller value of  $m$  is taken and the procedure repeated. It is easily verified that an efficient binary search can be organized around the fact that an appropriate  $m$  must exist in the range  $\left[ \frac{1}{2T} \sum_{i=1}^n s(p_i), \frac{1}{T} \sum_{i=1}^n s(p_i) \right]$ . The corresponding algorithm has the asymptotic bound  $R_{LFD}^\infty = 4/3$ , thus improving on the bound for *FFD*[ $C$ ].

An experimental analysis of average case behavior for these algorithms, plus a probabilistic analysis of *NF* analogous to that in [72] can be found in [2].

*Maximizing the minimum bin level* — This dual to the capacity minimization problem was studied by Deurmeyer, Friesen and Langston [37]. Clearly, it is also closely related to the problem of minimizing the difference between maximum and minimum bin levels, which was discussed in the previous section. In [37] it is shown that the *LFD* rule has a  $4/3$  bound (using the inverted ratio) for the max-min problem, just as it has for the min-max problem. As might be expected, set-differencing algorithms are also effective for this problem, but algorithms such as *MULTIFIT* can perform very poorly.

## 6. Vector Packing

In this section we consider one way of generalizing the classical one-dimensional bin packing problem to higher dimensions. Instead of each  $s(p)$  being a single number, we consider the case when it is a  $d$ -dimensional vector  $s(p) = \langle s_1(p), s_2(p), \dots, s_d(p) \rangle$ . The bin capacity is also a

$d$ -dimensional vector  $\langle C, C, \dots, C \rangle$ , and the goal is to pack the items in a minimum number of bins, given that the contents of any given bin must have vector sum less than or equal to its capacity. This problem models multiprocessor scheduling of unit-length tasks in the case when there are  $d$  resources, rather than just one as we assumed before. For simplicity we have normalized the amounts of resources available so that all  $d$  bounds are the same.

Note that the two-dimensional version of this problem is not the same as the problem of packing rectangles (to be discussed in the next section). A vector  $\langle s_1(p), s_2(p) \rangle$  could be thought of as representing a rectangle with length  $s_1(p)$  and width  $s_2(p)$ , and a bin of capacity  $\langle C, C \rangle$  as a square into which the rectangles are to be packed. However, the only types of packings allowed here would correspond to ones in which the rectangles were placed corner to corner, diagonally across the bin.

In [84], Kou and Markowsky show that any "reasonable" algorithm, i.e., one which does not yield packings in which the contents of two non-empty bins can be combined into a single bin, obeys the bound  $R_A \leq d+1$ , where  $d$  is the number of dimensions (an alternative proof can be found in [48], although there the theorem is not stated in its full generality). We note in passing that, in spite of the obvious desirability of the above "reasonable" property, not all the algorithms we have mentioned so far are reasonable — an obvious offender is NEXT FIT. However, FIRST FIT, FIRST FIT DECREASING, and many others *are* reasonable and hence do obey the above-mentioned, not very impressive (when  $d$  is large), bound. They are, in fact, better than reasonable, but not by as much as one would like. In [50], Garey, Graham, Johnson, and Yao analyze the  $d$ -dimensional problem and appropriate adaptations of *FF* and *FFD* to this multi-dimensional case (in *FFD* the items are sorted in non-decreasing order by the maximum components of their size vectors). They show that  $R_{FF}^\infty = d+7/10$ , which reduces to the familiar 17/10 result in the one-dimensional case, and that  $d \leq R_{FFD}^\infty \leq d+1/3$ . To date, no one has found any polynomial time approximation algorithm for the general  $d$ -dimensional problem with  $R_A^\infty < d$ . Yao has shown

[108] that any algorithm that is *faster* than *FF* or *FFD*, i.e., that has a running time that is  $o(n \log n)$  in the decision tree model of computation, *must* have  $R_A^\infty \geq d$ . Within this constraint, Fernandez de la Vega and Leuker [40] show, by extending their results in one dimension, that a polynomial time approximation scheme exists for the vector problem as well; i.e. there is a linear-time algorithm for finding solutions within  $\epsilon$  of  $d$  times the optimal. (The earlier caveats concerning running time apply here as well, however.) In spite of these results, we should note that the extensive simulation results of Maruyama, Chang, and Tang [96] for *FF*, *FFD*, and a variety of other algorithms indicate that average case behavior may not be nearly so bad here as the worst case bounds.

In the variant on this problem in which a partial order is present, however, things definitely get worse. Suppose that the set of items has a partial order  $\leq$  associated with it that constrains the allowable packings as in Section 3 (the multiprocessor scheduling rather than the assembly line balancing case). In this case the natural generalization of the ORDERED FIRST FIT DECREASING algorithm of Section 3 can be shown [50] to obey  $(1.691)d+1 \leq R_{OFFD}^\infty \leq (1.7)d+1$ , a definite worsening of our bounds when no partial order was present (the result mentioned in Section 3 for the one-dimensional version of this problem is a special case of this result). Similar results are obtained for the algorithm ORDERED FIRST FIT BY LEVEL, which works the same way as OFFD, except that instead of ordering items by non-increasing maximum size component, they are ordered by non-increasing "level" in the partial order [24]:  $R_{OFFL}^\infty = (1.7)d+1$ . That some type of pre-ordering is necessary for even this standard of performance follows from the fact that the algorithm without any pre-ordering, ORDERED FIRST FIT, has  $R_{OFF}^\infty = \infty$  [50].

## 7. Rectangle Packing

In this section we consider an active area of bin packing research: the problem of packing

rectangles into two-dimensional bins. The first version of this problem to be studied from a performance guarantee point of view is due to Baker, Coffman, and Rivest [8], and models a variety of problems, from computer scheduling to stock cutting. In this version, the items  $p_i$  are rectangles, with height  $h_i$  and width  $w_i$ . The goal is to pack them in a vertical strip of width  $C$ , so as to minimize the total height of the strip needed. The rectangles must be packed orthogonally, that is, no rotations are allowed: all rectangles must have their width parallel to the bottom of the strip.

The orthogonality restriction is justified on the basis of the proposed application to scheduling. Here the items once again correspond to tasks. The height of an item is the amount of processing time it requires, and its width is the amount of contiguous memory it needs. The strip width  $C$  is then the total memory available; the strip length is the amount of time needed to schedule all the items. In this application it makes no sense to rotate a rectangle, even by ninety degrees, since execution time is not in general directly translatable into a memory requirement.

Applications to stock-cutting occur in a variety of industrial settings where the "raw" material involved comes in rolls, for instance rolls of paper, rolls of cloth, rolls of sheet metal, etc. From these rolls we may wish to cut patterns (for labels, clothes, boxes, etc.) or merely just shorter, narrower rolls. In the simplest case, we can view the objects we wish to cut from the rolls as being, or approximating, rectangles. We minimize our wastage if we minimize the amount of roll (the strip length) used. Once again some form of orthogonality may be justified, since in many applications, the cutting is done by blades that must be either parallel or perpendicular to the strip, and the material may have a bias that dictates the orientation of the rectangles. However, ninety degree rotations may in some cases be allowable, and we will later say a bit about how the results we discuss can be extended to take this into account.

Because of the economic importance of efficient stock-cutting, a broad range of classical heuristic and enumerative methods have been applied in the last 20 years. For example, solution techniques

have been designed around linear programming, dynamic programming, branch and bound, network flow and heuristic search methods. (See [1,10,19,57] for such studies and references to a number of others.) The performance of these solution techniques is normally evaluated experimentally, rather than analytically, so they fall outside the scope of this survey.

In [8], Baker, Coffman, and Rivest consider a variety of strip packing algorithms based on a "bottom up – left justified" (BOTTOM-LEFT for short) packing rule. In a BOTTOM-LEFT packing, items are packed in turn, each item being placed as near to the bottom of the strip as it will fit and then as far to the left as it can be placed at that bottom-most level. Note that there is a difference in kind between two-dimensional packing rules, such as the BOTTOM-LEFT rule, and one-dimensional rules such as FIRST FIT and NEXT FIT. In the one-dimensional case there always exists an ordering of the items such that FIRST FIT (NEXT FIT) constructs an optimal packing. However, as shown in [8] this is not the case for BOTTOM-LEFT. In fact, Brown [13] has constructed instances in which the best BOTTOM-LEFT packing possible still yields a strip whose height is  $5/4$  times optimal.

However, although no preordering of the items may be able to yield an optimal packing, some may still be better than others. Various BOTTOM-LEFT algorithms can be considered, depending on how (if at all) the set of rectangles is initially preordered. It turns out that only one of the standard orderings seems to make a difference as far as worst case behavior is concerned. If we let *BL* stand for the simple BOTTOM-LEFT algorithm, and *BLIW*, *BLIH*, *BLDW*, and *BLDH* stand for the algorithm with preordering by increasing (i.e., non-decreasing) width, increasing height, decreasing width, and decreasing height, then we have

$$R_{BL} = R_{BLIW} = R_{BLIH} = R_{BLDH} = \infty; \quad R_{BLDW} = 3$$

For the special case of squares ( $h_i = w_i$  for all  $p_i$ ), the *BLDH* algorithm becomes equivalent to *BLDW*, and the result improves to  $R_{BLDW} = 2$  [8].

For the case of arbitrary rectangles, subsequent work has yielded some improvements. The FIRST FIT DECREASING HEIGHT "level" algorithm of Coffman, Garey, Johnson, and Tarjan [29] (to be described later) can be shown to have  $R_{FFDH} = 2.7$ , and an algorithm of Sleator [104] further reduced the bound to  $R_A = 2.5$ .

These results all concern *absolute* worst case performance ratios. Indeed, for this problem it would again seem as if absolute and asymptotic performance ratios should be equivalent, since heights can be scaled to arbitrarily large values. However, such scaling may not be sensible in many practical applications, where some strict upper bound on height may be imposed. In this case, asymptotic analysis may be a more meaningful measure, giving us guarantees that hold as the optimal strip length becomes very large with respect to this maximum possible item height. As might be expected, these asymptotic guarantees can be better than the absolute ones (although they do not equal 1, as they would if all rectangle widths were equal, thus reducing us to the capacity minimization problem of Section 4). For instance,  $R_{BLDW}^{\infty} = 2$ , an improvement of 1 over the absolute guarantee for *BLDW*, but a long way from optimal.

The search for strip packing algorithms with better asymptotic worst case ratios was taken up in [29] by Tarjan and ourselves. The new algorithms were based on a different type of packing rule, suggested by Golan [58], and were called "level" algorithms. These algorithms involve an attempt to apply our old knowledge about one dimensional bin packing. Note that if all rectangles have the same height, the two-dimensional problem essentially reduces to the one-dimensional case: in an optimal packing the items may be placed in rows or "levels." Each level in the packing then corresponds to a bin and the height of the packing corresponds to the number of bins used. The basic idea of a level algorithm is the following: First, the items are preordered by non-increasing height. The packing is then constructed as a sequence of *levels*, each rectangle being placed so that its bottom rests on one of these levels. The first level is simply the bottom of the bin. Each subsequent level is defined by a horizontal line drawn through the top of the tallest rectangle on the

previous level. This is best illustrated by considering the two basic level algorithms proposed in [29].

In the algorithm NEXT FIT DECREASING HEIGHT rectangles are packed left-justified on a level until the next rectangle will not fit, in which case it is used to start a new level above the previous one, on which the packing proceeds. Note the analogy with the one-dimensional NEXT FIT algorithm. In the FIRST FIT DECREASING HEIGHT algorithm (another analog), each rectangle is placed left-justified on the first (i.e., lowest) level in which it will fit. If none of the current levels has room, a new one is started as with the *NFDH* algorithm. See Figure 6 for an example of an *FFDH* packing.

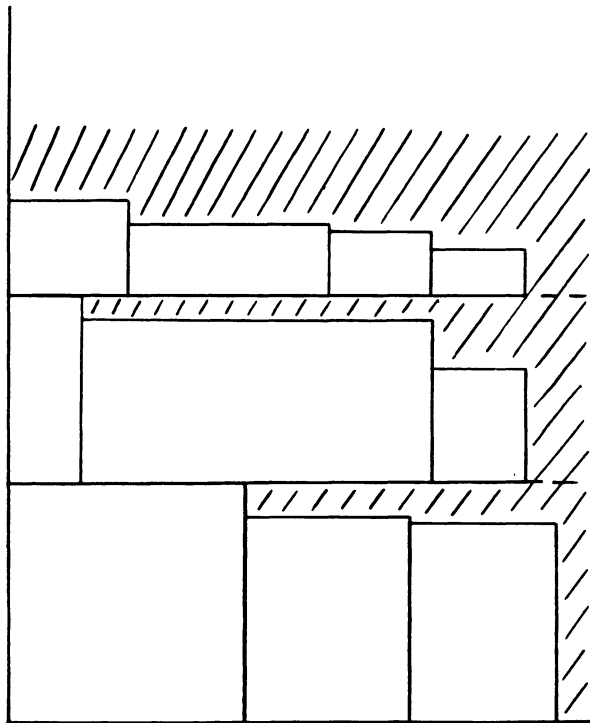


Figure 6. An example of FFDH packing.

At first glance, one would expect *NFDH* and *FFDH* to be worse than their one-dimensional counterparts, given all the space that may be wasted in a level above items which are shorter than the first one. However, it turns out that this wasted space is strictly bounded, and by a collapsing sum argument it can be concluded that, exactly as in the one-dimensional case,  $R_{NFDH}^{\infty} = 2$  and  $R_{FFDH}^{\infty} = 1.7$ . The results for bounded item widths also resemble their one-dimensional counterparts.

For the special case of squares the asymptotic worst case ratio is reduced to 1.5. The INCREASING rules *BLIH* (or *BLIW*) and *NFIH* have also been analyzed for this special case. In [6] Baker, Calderbank, Coffman and Lagarias show that  $R_{BLIH}^{\infty} = R_{NFIH}^{\infty} = 1.691$ , by extending the results in [7]. These rules will be discussed later in connection with another application.

We should note that level-by-level packings have a special significance stemming from their relation to *guillotine cuts* [57]. Guillotine cuts are edge-to-edge cuts of a rectangle or strip parallel to its length or width. The 3-stage guillotine cuts corresponding to level-by-level packings such as Figure 6 involve first a set of horizontal guillotine cuts, then a set of vertical cuts and finally another set of horizontal "trim" cuts. The special constraints of guillotine cuts apply in several applications, the chief one usually mentioned being the cutting of rectangular plates of glass.

Returning to the general problem, let us consider where further improvements might be found. Further orderings by size to approach the *FFD* performance in one-dimension do not appear feasible; *FFD* requires that the items be preordered by non-increasing size, which here corresponds to non-increasing width, and since *FFDH* already requires items to be preordered by height, any additional preordering becomes impossible. Fortunately, there are ways of approximating *FFD* in the two-dimensional case. In the *SPLIT FIT* algorithm of [29], the set of rectangles is partitioned into two parts, those with width exceeding  $C/2$  and those without, and each subset is ordered by non-increasing height. Packings for the two sets are then combined in an involved manner, and the

result is an algorithm with  $R_{SF}^\infty = 1.5$ . This idea of splitting the set of rectangles into subsets according to width can be carried even further. In [59] Golan described an algorithm for which  $R_A^\infty \leq 4/3$ , and in [5] Baker, Brown, and Katseff devised a much more complicated algorithm for which  $R_A^\infty \leq 5/4$ , a bound which is close to the  $11/9$  guarantee provided by *FFD* in the one-dimensional case.

So far, all the rectangle packing algorithms we have discussed for which  $R_A^\infty \leq \infty$  have involved some preordering of the rectangles, and hence are not "on-line" algorithms. However, such algorithms might well be required in scheduling applications, and so the question of finding an on-line algorithm with reasonable worst case behavior becomes relevant. Baker and Schwartz, in [9], show that such algorithms exist by devising what they call "shelf" algorithms. These are variants on the level algorithms above in which levels, rather than being determined by their tallest item, come in fixed sizes. If we assume that 1 is an *a priori* upper bound on rectangle height, the standard levels will come in heights  $r^{-k}$ ,  $k \geq 0$ , for some prespecified value of  $r$ ,  $0 < r < 1$ . Whenever a rectangle  $p_i$  is to be packed in the NEXT FIT SHELF( $r$ ) algorithm, one first determines that value of  $k$  such that  $r^{k+1} < h_i \leq r^k$ . If there is a level of height  $r^k$  already in the packing, and  $p_i$  will fit in the currently active one, it is placed there. Otherwise it is placed in a new such level, which becomes the currently active one for that height. The algorithm FIRST FIT SHELF( $r$ ) is defined analogously.

Although these shelf algorithms clearly have considerable space-wasting potential, it turns out that the wastage can once again be bounded, and in fact  $R_{NFS(r)}^\infty = 2/r$  and  $R_{FFS(r)}^\infty = 1.7/r$ . Note that these approach the values for *NFDH* and *FFDH* as  $r$  approaches 1. However, as  $r$  approaches 1 the amount of wastage to be expected in small examples increases as  $C/(1-r)$ , and so a trade-off is involved. The best absolute worst case ratio is obtained by *FFS*( $r$ ) when  $r \sim .622$ , in which case we have  $R_{FFS(r)} \sim 6.9863$ .

The limitations inherent in the on-line approach are investigated by Brown, Baker and Katseff in [15]. They show that *any* on-line algorithm must obey  $R_A \geq 2$ . (The paper also contains bounds for on-line algorithms in the special case where they happen to be given sets of rectangles in some sorted order, but must still pack each item in turn, without being able to look ahead or to move an item once it is placed).

As a final contribution to strip packing we mention the results of Coffman and Gilbert [31] on *dynamic* packings in two dimensions. This is the natural extension of dynamic storage allocation in one dimension as defined in Section 3. In [31] the problem is specialized to squares and bottom up packings in a strip of width  $w$ . Extending the definitions in the one dimensional problem, let the squares in list  $L$  have a maximum size of  $j \times j$  and assume that the total area of packed squares never exceeds  $mw$ . Let  $BL(L)$  be the maximum height achieved by squares in  $L$  under the BOTTOM LEFT algorithm. For the asymptotic bound

$$C_{BL}^{\infty}(j) = \lim_{\substack{m \rightarrow \infty \\ w \rightarrow \infty}} \sup_L \frac{BL(L)}{m},$$

it is proved in [31] that

$$C_{BL}^{\infty}(j) \leq \frac{H_j}{\log 2 - \frac{1}{2}} \approx 5.177 \log j,$$

where  $H_j$  is the  $j$ th harmonic number. Moreover, lists are given which show that  $C_{BL}^{\infty}(j) = \Theta(\log j)$ . A number of results for finite  $m$  and  $w$  are given in [31] along with extensions to more than two dimensions.

There have been two papers to date that cover average case analysis for strip packing. In [42], Frederickson proposes an off-line algorithm combining *FFD* with specially tuned shelf sizes and specifically designed for the case when item sizes and widths are independently and uniformly distributed between  $O$  and  $C$ . Although the expected wastage may be large in absolute terms

(proportional to  $n^{3/4}$ ), the ratio of expected strip length to a lower bound on the optimal length (obtained by dividing the expected total area of rectangles by the strip width  $C$ ) approaches 1 as  $n$  goes to infinity.

In [67], Hofri concentrates more on the on-line case, extending his earlier work with Coffman, So and Yao [32] on the expected behavior of one dimensional NEXT FIT to the strip packing problem. He considers two new on-line algorithms. The first is a level algorithm in which there is no initial reordering of the list of items, and hence the height of a level is not determined by its first rectangle but by the tallest, whichever one that might be. Otherwise the packing rule is basically a NEXT FIT one: an item is packed in the current level unless it cannot fit along the bottom, in which case it starts a new level, whose bottom is coincident with the top of the tallest item in the earlier level. Hofri calls this algorithm NEXT FIT, as opposed to NEXT FIT DECREASING HEIGHT where the items are preordered. Hofri's other new on-line algorithm is appropriate in the case where ninety degree rotations are allowed, and is called ROTATABLE NEXT FIT. This algorithm is the same as NEXT FIT except that each item is rotated before it is packed so that its height does not exceed its width.

Both of these two new on-line algorithms have  $R_A^\infty = \infty$  and so are not very attractive from a worst case point of view. However, Hofri shows that when heights and widths are independent and uniformly distributed between  $O$  and  $C$ , they are not that much worse than NEXT FIT DECREASING HEIGHT, which has  $R_{NFDH}^\infty = 2$  and is not an on-line algorithm. As  $n \rightarrow \infty$ , Hofri's results indicate that NEXT FIT DECREASING HEIGHT averages roughly  $4/3$  times the above-mentioned lower bound on optimal strip length. ROTATABLE NEXT FIT is only slightly worse, and NEXT FIT's ratio is only about  $3/2$ .

Having introduced the case where ninety degree rotations are allowed, we should mention that some of the worst case results mentioned above also apply to this case, in that the values of  $R_A$  and

$R_A^\infty$  are unchanged if such rotations are allowed in the construction of optimal packings. This holds true in particular for *NFDH* and *BLDW*, since the proofs of the bounds for these algorithms are based on pure area arguments. So far no algorithm has been found that attains improved guarantees by actually using such rotations itself, and the results mentioned above for the performance of strip packing algorithms when all items are squares (and hence ninety degree rotation cannot help) indicate that we can expect only limited improvements.

A rectangle packing problem closely related to strip packing is that of packing a given set of rectangles into an enclosing rectangle of minimum area. Strip packing is the special case where the width is fixed. In this general problem both length and width are allowed to vary. To date there has not been much work on this problem from a performance guarantee point of view. Two papers of interest have addressed the case when all the items to be packed are squares. In [81], Kleitman and Krieger show that a collection of squares whose total area is unity can always be packed into a rectangle with area  $4/\sqrt{6}$ , and this is the minimum area for which such a packing is guaranteed. Furthermore, a  $2/\sqrt{3}$  by  $\sqrt{2}$  rectangle is the unique rectangle that will always suffice. In [39], Erdős and Graham consider the minimum sized square required to contain a collection of unit squares, and show that this size can be nontrivially decreased if rotations other than ninety degrees are allowed.

Approximation algorithms for a problem complementary to these have been studied by Baker, Calderbank, Coffman and Lagarias [6]. Their problem was to pack the maximum number of squares from a given list into a rectangle of fixed dimensions. They analyzed both a BOTTOM UP INCREASING and a level-by-level NEXT FIT INCREASING algorithm and proved that  $R_{BUI}^\infty = R_{NFI}^\infty = 4/3$ , where again these are based on the inverted ratios  $OPT(L)/A(L)$ .

It must be pointed out that there is a sizable literature on square packing which we shall not survey here, primarily because it does not concern results closely related to approximation

algorithms. The following list illustrates informally the variety of questions that have been asked: What is the smallest square sufficient to enclose  $n$  unit squares, rotations allowed? Can a rectangle with integer sides be tiled by a sequence of consecutive squares with sides  $1, 2, 3, \dots$ ? What is the smallest number of squares with integer sides into which a given square with integer sides can be partitioned? Extensive discussions of these and other intriguing but difficult questions can be found in [47, 101], along with discussions of similar problems in packing circles and spheres.

The final rectangle packing problem we shall consider is a straightforward generalization of the one-dimensional case. Here the problem is once again simply to minimize the number of bins used, the bins now being large rectangles of some fixed dimensions into which the given set of rectangles must be packed. We first note that if the number of possible rectangle sizes is sufficiently small, a Gilmore-Gomory style linear programming approach can be applied [57] with useful results. For the general problem, the only algorithm which to date has been analyzed from the worst case point of view is a composite algorithm proposed by Chung, Garey and Johnson [20]. We shall denote this algorithm by "*FFDH*•*FFD*," as it is based on the algorithm *FFDH* for strip packing and *FFD* for one-dimensional bin packing. The idea of the algorithm is as follows: Suppose the standard bin has width  $W$  and height  $H$ . First use the *FFDH* algorithm to pack the set of rectangles into a strip of width  $W$ . Next, decompose this packing into blocks corresponding to the levels created by *FFDH*. Each block can be viewed as a rectangle of width  $W$  and height the height of the level. Thus, packing these blocks into rectangular bins of width  $W$  becomes a simple one-dimensional bin packing problem, where the size of an item (block) is its height. Apply *FFD* to this one-dimensional problem.

The analysis of this algorithm in [20] shows that  $2.022 \leq R_{FFDH \cdot FFD}^{\infty} \leq 2.125 \dots$ . Note that this leaves open the interesting possibility that  $R_{FFDH \cdot FFD}^{\infty} = (R_{FFDH}^{\infty})(R_{FFD}^{\infty}) = (17/10)(11/9) = 2.0777\dots$ , although the proof of such a result might well represent quite a challenge.

Quite recently Karp, Luby and Spaccamela [79] have considered a probability model of this problem under the assumption that the enclosing rectangles are unit squares. The dimensions of each of the  $n$  rectangles to be packed are independent random variables uniformly distributed over  $[0,1]$ . They devise an approximation algorithm requiring  $n/4 + O(n^{1/2} \log n)$  bins on the average, thus generalizing the one-dimensional results in [76] and Frederickson's [42] strip packing results. Since the expected total area of the  $n$  rectangles is  $n/4$ , the ratio of the expected number of bins required by their algorithm to the expected number in a perfect packing approaches 1 as  $n \rightarrow \infty$ . Similar results are derived for extensions to more than two dimensions.

## 8. Directions for Future Research

In this section we briefly mention some of the open problems that we feel are significant from either a mathematical or practical point of view. First there is the basic problem of finding simpler and more general proof techniques. Although we have concentrated here on results rather than proof techniques, most of the results we have cited have only been proved by very problem-specific techniques that have rarely been exploited in analyzing related problems. It is true that researchers have been able to use intuition gained in studying the classical one-dimensional case in deriving results for the more complicated variants and generalizations, but unfortunately this is not often very apparent in the resulting proofs. The closest to a general method for proving results of this sort is the "weighting function" approach, as noted in Section 2, but so far the details of how this approach is used vary considerably from one problem to the next.

On a less fundamental level, there is of course the problem of finding better algorithms for the various problems, especially in the area of rectangle packing, and of tightening up the bounds on the algorithms already proposed but incompletely analyzed. There is also always room for new bin packing variants, the key being to find a variant that models practical problems *and* is susceptible to meaningful analysis. For instance, questions are often asked about the case when there are different

types of bins (i.e., different sizes, different costs, etc.). The only work that can be cited here appears to be that of Langston [87] mentioned in Section 5.

Another technical problem is the very fundamental one of lower bounds. We have mentioned a number of lower bound results for on-line algorithms and we have cited the important recent developments in fully polynomial approximation schemes by Fernandez de la Vega and Lueker [40] and Karmarkar and Karp [74]. But the question of such schemes for two dimensional packings remains open, as does the question: Is there a polynomial time algorithm for the one dimensional problem that always comes within some additive constant of the optimum?

Our final remarks concern probabilistic analysis. We have noted the impressive increase in this research in the past few years. The most far-reaching contributions appear to have been those establishing rates of convergence to optimality of certain approximation algorithms, in the sense of expected performance. These results have usually been in the form of bounds that leave considerable room for future improvements. There is the added challenge, of course, to provide measures of second moments, and indeed, distributions of objective functions.

A characteristic weakness of many of the results to date has been that the algorithms analyzed have been chosen for their mathematical tractability rather than their attractiveness from a practical point of view. A prominent open problem is still a satisfactory average-case analysis of FIRST FIT, not to mention FIRST FIT DECREASING. A new such problem is to extend the  $O(n^{-\log n})$  performance estimate to the more natural largest-pair-first set-differencing algorithm [75] introduced in Section 4. In view of the methods currently available, expecting exact results may be unreasonable. However, good bounds and results for interesting special cases would appear to be well worth the effort.

## 9. References

- [1] Albano, A. and Sapuppo, G., "Optimal allocation of two-dimensional irregular shapes using heuristic search methods," *IEEE Trans. Syst., Man, Cybern.*, SMC-10 (1980), 242-248.
- [2] Assmann, S. B., Doctoral Dissertation, Department of Mathematics, M.I.T., Cambridge, Mass. (1983).
- [3] Assmann, S. B., Johnson, D. J., Kleitman, D. J. and Leung, J. Y-T., "On a dual version of the one-dimensional bin packing problem," *J. of Algorithms* (to appear).
- [4] Baker, B. S., "A new proof for the first-fit decreasing bin-packing algorithm," Technical Memorandum (1983), Bell Laboratories, Murray Hill, N.J. 07974.
- [5] Baker, B. S., Brown, D. J., and Katseff, H. P., "A  $5/4$  algorithm for two-dimensional packing," *J. of Algorithms*, 2 (1981), 348-368.
- [6] Baker, B. S., Calderbank, A. R., Coffman, E. G., Jr., and Lagarias, J. C., "Approximation algorithms for maximizing the number of squares packing into a rectangle," *SIAM J. of Alg. Disc. Meth.* (to appear).
- [7] Baker, B. S. and Coffman, E. G., Jr., "A tight asymptotic bound for next-fit-decreasing bin-packing," *SIAM J. Alg. Disc. Meth.* 2 (1981), 147-152.
- [8] Baker, B. S., Coffman, E. G., Jr., and Rivest, R. L., "Orthogonal packings in two dimensions," *SIAM J. Comput.* 9 (1980), 846-855.
- [9] Baker, B. S. and Schwarz, J. S., "Shelf algorithms for two-dimensional packing problems," *SIAM J. Comput.* (to appear).
- [10] Biro, M. and Boros, E., "A network flow approach to non-guillotine cutting problems," *Working Paper MO/30* (1982), Computer and Automation Institute, Hungarian Academy of Sciences, Budapest.

- [11] Brown, A. R., *Optimum Packing and Depletion*, American Elsevier, New York (1971).
- [12] Brown, D. J., "A lower bound for on-line one-dimensional bin packing algorithms," Technical Report R-864 (1979), Coordinated Science Laboratory, University of Illinois, Urbana, IL.
- [13] Brown, D. J., "An improved BL lower bound," *Inf. Proc. Letters* 11 (1980) 37-39.
- [14] Brown, D. J., private communication (1980).
- [15] Brown, D. J. and Baker, B. S. and Katseff, H. P., "Lower bounds for the on-line two-dimensional packing algorithms," *Acta Informatica*, 18 (1982), 207-225.
- [16] Bruno, J. L. and Downey, P. J., "Probabilistic bounds on the performance of list scheduling," *Tech. Rep. TR 82-19*, Computer Science Dept., University of Arizona, Tucson, Ariz.
- [17] Chandra, A. K. and Wong, C. K., "Worst-case analysis of a placement algorithm related to storage allocation," *SIAM J. Comput.* 4 (1975), 249-263.
- [18] Chandra, A. K., Hirschberg, D. S., and Wong, C. K., "Bin packing with geometric constraints in computer network design," Computer Science Research Report RC 6895 (1977), IBM Research Center, Yorktown Heights, New York.
- [19] Christofides, N. and Whitlock, C., "An algorithm for two-dimensional cutting problems," *Oper. Res.* 25 (1977), 30-44.
- [20] Chung, F. R. K., Garey, M. R. and Johnson, D. J., "On packing two-dimensional bins," *SIAM J. Alg. Disc. Meth.* 3 (1982), 66-76.
- [21] Cody, R. A. and Coffman, E. G., Jr., "Record allocation for minimizing expected retrieval costs on drum-like storage devices," *Journal of the ACM* 23 (1976), 103-115.
- [22] Coffman, E. G., Jr., "An introduction to proof techniques for packing and sequencing algorithms," in *Deterministic and Stochastic Scheduling*, M.A.H. Dempster, et al. (eds.),

- (1982), 245-270, Reidel Publishing Co., Amsterdam.
- [23] Coffman, E. G., Jr., "An introduction to combinatorial models of dynamic storage allocation," *SIAM Review* (to appear).
- [24] Coffman, E. G., Jr., Frederickson, G. and Lueker, G. S., "A note on expected makespans for largest-first sequences of independent tasks on two processors," *Math of OR* (to appear).
- [25] Coffman, E. G., Jr., Frederickson, G. N. and Lueker, G. S., manuscript in preparation.
- [26] Coffman, E. G., Jr., Garey, M. R., and Johnson, D. S., "An application of bin-packing to multiprocessor scheduling," *SIAM J. Comput.* 7 (1978), 1-17.
- [27] Coffman, E. G., Jr., Garey, M. R., and Johnson, D. S., "Dynamic bin packing," *SIAM J. Comput.* 12 (1983), 227-258.
- [28] Coffman, E. G., Jr., Garey, M. R. and Johnson, D. S., "Performance of packing algorithms for divisible sequences of item sizes," paper in preparation.
- [29] Coffman, E. G., Jr., Garey, M. R., Johnson, D. S., and Tarjan, R. E., "Performance bounds for level-oriented two-dimensional packing algorithms," *SIAM J. Comput.* 9 (1980), 808-826.
- [30] Coffman, E. G., Jr. and Gilbert, E. N., "On the expected relative performance of list scheduling," Technical Memorandum, Bell Laboratories, Murray Hill, N.J. 07974 (1983).
- [31] Coffman, E. G., Jr. and Gilbert, E. N., "Dynamic first-fit packings in two or more dimensions," Technical Memorandum, Bell Laboratories, Murray Hill, N.J. 07974 (1983).
- [32] Coffman, E. G., Jr., Hofri, M., So, K., and Yao, A. C., "A stochastic model of bin packing," *Inf. and Control* 44 (1980), 105-115.
- [33] Coffman, E. G., Jr., and Leung, J. Y., "Combinatorial analysis of an efficient algorithm for

- processor and storage allocation," *SIAM J. Comput.* 8 (1979), 202-217.
- [34] Coffman, E. G., Jr., Leung, J. Y., and Ting, D. W., "Bin packing: maximizing the number of pieces packed," *Acta Informatica* 9 (1978), 263-271.
- [35] Coffman, E. G., Jr. and Sethi, R., "A generalized bound on LPT sequencing," *RAIRO-Informatique* 10 (1976), 17-25.
- [36] Dempster, M. A. H., Fisher, M. L., Jansen, L., Lageweg, B. J., Lenstra, J. K. and Rinnooy Kan, A. H. G., "Analysis of heuristics for stochastic programming: Results for Hierarchical scheduling problems," *Operations Res.*, 29 (1981), 707-716.
- [37] Deuermeier, B. L., Friesen, D. K. and Langston, M. A., "Maximizing the minimum processor finish time in a multiprocessor system," *SIAM J. Alg. Disc. Meth.* 3 (1982), 190-196.
- [38] Easton, M. C. and Wong, C. K., "The effect of a capacity constraint on the minimal cost of a partition," *J. Assoc. Comput. Mach.* 22 (1975), 441-449.
- [39] Erdős, P. and Graham, R. L., "On packing squares with equal squares," *J. Combinatorial Theory Ser. A* 19 (1975), 119-123.
- [40] Fernandez de la Vega, W. and Lueker, G. S., "Bin packing can be solved within  $1+\epsilon$  in linear time," *Combinatorica* 1 (1981), 349-355.
- [41] Finn, G. and Horowitz, E., "A linear time approximation algorithm for multiprocessor scheduling," *BIT* 19 (1979), 312-320.
- [42] Frederickson, G. N., "Probabilistic analysis for simple one- and two-dimensional bin packing algorithms," *Inf. Proc. Letters* 11 (1980), 156-161.
- [43] Frenk, H. and Rinnooy Kan, A. H. G., "The asymptotic optimality of the LPT heuristic," Erasmus University, Rotterdam, The Netherlands, (to be published).

- [44] Friesen, D. K., "Sensitivity analysis for heuristic algorithms," Technical Report UIUCDCS-R-78-939 (1978), Dept. Comp. Sci., Univ. of Illinois, Urbana, IL. *SIAM J. Comput.*, (to appear).
- [45] Friesen, D. K. and Langston, M. A., "Analysis of a compound bin-packing algorithm," (to appear).
- [46] Galambos, G. and Turan, G., Laboratory of Cybernetics, Josef Attila University, Szeged, Hungary (private communication).
- [47] Gardner, M., "Some packing problems that cannot be solved by sitting on the suitcase," in Mathematical Games column, *Scientific American*, Oct. 1979, 18-26.
- [48] Garey, M. R. and Graham, R. L., "Bounds on multiprocessor scheduling with resource constraints," *SIAM J. Comput.* 4 (1974), 187-200.
- [49] Garey, M. R., Graham, R. L., and Johnson, D. S., "On a number-theoretic bin packing conjecture," *Proc. 5th Hungarian Combinatorics Colloquium*, North-Holland, Amsterdam (1978), 377-392.
- [50] Garey, M. R., Graham, R. L., Johnson, D. S. and Yao, A. C., "Resource constrained scheduling as generalized bin packing," *J. Combinatorial Theory Ser. A* 21 (1976), 257-298.
- [51] Garey, M. R. and Johnson, D. S., "Approximation algorithms for combinatorial problems: an annotated bibliography," in J. F. Traub (ed.), *Algorithms and Complexity: New Directions and Recent Results*, Academic Press, New York (1976), 41-52.
- [52] Garey, M. R. and Johnson, D. S., *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman and Co., San Francisco (1979).
- [53] Garey, M. R. and Johnson, D. S., "Approximation algorithms for bin-packing problems — A survey," in *Analysis and Design of Algorithms in Combinatorial Optimization*, G. Ausiello

and M. Lucertini (eds.), Springer-Verlag, New York, 1981, 147-172.

- [54] Garey, M. R. and Johnson, D. S., paper in preparation.
- [55] Gilmore, P. C. and Gomory, R. E., "A linear programming approach to the cutting stock problem," *Operations Res.* 9 (1961), 849-859.
- [56] Gilmore, P. C. and Gomory, R. E., "A linear programming approach to the cutting stock program — Part II," *Operations Res.* 11 (1963), 863-888.
- [57] Gilmore, P. C. and Gomory, R. E., "Multistage cutting stock problems of two and more dimensions," *Operations Res.* 13 (1965), 94-120.
- [58] Golan, I., "Two orthogonal oriented algorithms for packing in two dimension," Report 1979/311/MHM, Computer Center M.O.D., P. O. Box 2250, Haifa, Israel (1979).
- [59] Golan, I., "Performance bounds for orthogonal, oriented two-dimensional packing algorithms," *SIAM J. Comput.* 10 (1981), 571-582.
- [60] Graham, R. L., "Bounds for certain multiprocessing anomalies," *Bell System Tech. J.* 45 (1966), 1563-1581.
- [61] Graham, R. L., "Bounds for Dynamic Storage Allocation Strategies," Technical Memorandum, Bell Laboratories, Murray Hill, N.J. (1968).
- [62] Graham, R. L., "Bounds on multiprocessing timing anomalies," *SIAM J. Appl. Math.* 17 (1969), 263-269.
- [63] Graham, R. L., "Bounds on multiprocessing anomalies and related packing algorithms," *Proc. 1972 Spring Joint Computer Conference*, AFIPS Press, Montvale, N.J. (1972), 205-217.
- [64] Graham, R. L., "Bounds on performance of scheduling algorithms," in E. G. Coffman, Jr. (ed.), *Computer and Job-Shop Scheduling Theory*, John Wiley & Sons, New York (1976), 165-227.

- 
- [65] Graham, R. L., Lawler, E. L., Lenstra, J. K., and Rinnooy Kan, A. H. G., "Optimization and approximation in deterministic sequencing and scheduling: a survey," *Annals Disc. Math.* 5 (1979), 287-326.
  - [66] Hoffman, U., "A class of simple online bin packing algorithms," *Computing*, 29 (1982), 227-239.
  - [67] Hofri, M., "Two dimensional packing: expected performance of simple level algorithms," *Inf. and Control* 45 (1980), 1-17.
  - [68] Hofri, M., "Bin-packing: An analysis of the Next-Fit algorithm," *Tech. Rep. No. 242*, Dept. of Computer Science, The Technion, Haifa, Israel (1982).
  - [69] Johnson, D. S., "Near-optimal bin packing algorithms," Technical Report MAC TR-109 (1973), Project MAC, Massachusetts Institute of Technology, Cambridge, Mass.
  - [70] Johnson, D. S., "Fast algorithms for bin packing," *J. Comput. Syst. Sci.* 8 (1974), 272-314.
  - [71] Johnson, D. S., "The *NP*-completeness column: An ongoing guide," *J. of Algorithms* 2 (1981), 393-405 (and succeeding issues).
  - [72] Johnson, D. S., Demers, A., Ullman, J. D., Garey, M. R., and Graham, R. L., "Worst-case performance bounds for simple one-dimensional packing algorithms," *SIAM J. Comput.* 3 (1974), 299-325.
  - [73] Karmarkar, N., "Probabilistic analysis of some bin-packing problems," *Proc. 23rd Ann. Symp. on Foundations of Computer Science*, IEEE Computer Soc., Nov. 1982 (full paper to appear elsewhere).
  - [74] Karmarkar, N. and Karp, R. M., "An efficient approximation scheme for the one-dimensional bin packing problem," *Proc. 23rd Ann. Symp. on Foundations of Computer Science*, IEEE Computer Soc., Nov. 1982 (full paper to appear elsewhere).

- [75] Karmarkar, N. and Karp, R. M., "The differencing method of set partitioning," Computer Science Div., University of California, Berkeley, Calif., to be published.
- [76] Karmarkar, N., Karp, R. M., Lueker, G. S. and Murgolo, F., Computer Science Div., University of California, Berkeley, Calif., paper in preparation.
- [77] Karmarkar, N., Karp, R. M., Lueker, G. S., and Odlyzko, A., Bell Laboratories, Murray Hill, N.J., paper in preparation.
- [78] Karp, R. M., "Reducibility among combinatorial problems," in R. E. Miller and J. W. Thatcher (eds.), *Complexity of Computer Computations*, Plenum Press, New York (1972), 85-103.
- [79] Karp, R. M., Luby, M. G. and Spaccamela, A. M., "Probabilistic analysis of multidimensional bin packing problems," Computer Science Div., University of California, Berkeley, Calif., paper in preparation.
- [80] Kaufman, M. T., "An almost-optimal algorithm for the assembly line scheduling problem," *IEEE Trans. Computers* C-23 (1974), 1169-1174.
- [81] Kleitman, D. J. and Krieger, M. K., "An optimal bound for two dimensional bin packing," *Proc. 16th Ann. Symp. on Foundations of Computer Science*, IEEE Computer Society, Long Beach, CA (1975), 163-168.
- [82] Knödel, W., "A bin-packing algorithm with complexity  $O(n \log n)$  and performance 1 in the stochastic limit," *Proc., 10th Symp. on Math. Foundations in Comp. Sci.* (1981). (to appear in *Lecture Notes in Computer Science*, Springer-Verlag).
- [83] Knuth, D. E., *Fundamental Algorithms*, Vol. 1, Second edition, Addison-Wesley (1973).
- [84] Kou, L. T. and Markowsky, G., "Multidimensional bin packing algorithms," *IBM J. Res. & Dev.* 21 (1977), 443-448.

- [85] Krause, K. L., Shen, Y. Y., and Schwetman, H. D., "Analysis of several task-scheduling algorithms for a model of multiprogramming computer systems," *J. Assoc. Comput. Mach.* **22** (1975), 522-550.
- [86] Langston, M. A., *Processor Scheduling with Improved Heuristic Algorithms*, Doctoral dissertation, Texas A&M University, College Station, Texas (1981).
- [87] Langston, M. A., "Performance of bin-packing heuristics for maximizing the number of pieces packed into bins of different sizes," *Tech. Rep. No. CS-82-090*, Computer Science Dept., Washington State University, Pullman, Wash. (1982).
- [88] Langston, M. A., "Improved LPT scheduling for identical processor systems," *RAIRO-Technique et Science Informatiques*, **1** (1982), 69-75.
- [89] Langston, M. A., "Improved 0/1 interchange scheduling," *BIT* **22** (1982), 282-290.
- [90] Lee, C. C. and Lee, D. T., "A simple on-line packing algorithm," Dept. of Electrical Engineering and Computer Science (1983), Northwestern Univ., Evanston, Ill. 60201 (to appear).
- [91] Liang, F. M., "A lower bound for on-line bin packing," *Information Processing Lett.* **10** (1980), 76-79.
- [92] Loulou, R., "Tight bounds and probabilistic analysis of two heuristics for parallel processor scheduling," *Tech. Rep.*, Faculty of Management, McGill University, Montreal (1982).
- [93] Loulou, R., "Probabilistic behavior of optimal bin packing solutions," *Tech. Rep.*, Faculty of Management, McGill University, Montreal (1982).
- [94] Lueker, G. S., "An average-case analysis of bin packing with uniformly distributed item sizes," *Tech. Rep. No. 181* (1982), Dept. of Information and Computer Science, University of California, Irvine, CA 92717.

- [95] Lueker, G. S., "Bin packing with items uniformly distributed over intervals  $[a, b]$ ," Dept. of Information and Computer Science (1983), University of California, Irvine, CA 92717 (to be published).
- [96] Maruyama, K., Chang, S. K., and Tang, D. T., "A general packing algorithm for multidimensional resource requirements," *Internat. J. Comput. Infor. Sci.* **6** (1977), 131-149.
- [97] Ong, H. L., Magazine, M. J., and Wee, T. S., "Probabilistic analysis of bin-packing heuristics," *Operations Res.* (to appear).
- [98] Robson, J. M., "Bounds for some functions concerning dynamic storage allocation," *Journal of the ACM* **21** (1974), 491-499.
- [99] Robson, J. M., "Worst-case fragmentation of first-fit and best-fit storage allocation strategies," *Computer J.*, **20** (1977), 242-244.
- [100] Sahni, S., "Algorithms for scheduling independent tasks," *Journal of the ACM* **23** (1976), 116-127.
- [101] Schrijver, A. (ed.), *Packing and Covering in Combinatorics*, published by Mathematical Centre, Tweede Boerhaavestraat 49, Amsterdam (1979).
- [102] Shapiro, S. D., "Performance of heuristic bin packing algorithms with segments of random length," *Information and Control* **35** (1977), 146-148.
- [103] Shearer, J. B., "A counterexample to a bin packing conjecture," *SIAM J. Alg. Disc. Meth.* **2** (1981), 309-310.
- [104] Sleator, D. K. D. B., "A 2.5 times optimal algorithm for bin packing in two dimensions," *Information Processing Lett.* **10** (1980), 37-40.
- [105] Steele, M., Stanford University (private communication).

- [106] Taylor, D. B., "Container stacking: an application of mathematical programming," Draft (1979).
- [107] Wee, T. S. and Magazine, M. J., "Assembly line balancing as generalized bin-packing," *Operation Res. Letters*, 1 (1982), 56-58.
- [108] Wong, C. K. and Yao, A. C., "A combinatorial optimization problem related to data set allocation," *Rev. Francaise Automat. Informat. Recherche Operationelle Ser. Bleue* **10.5** (suppl.) (1976), 83-95.
- [109] Yao, A. C., "New algorithms for bin packing," *J. Assoc. Comput. Mach.* **27** (1980), 207-227.
- [110] Yue, P. C. and Wong, C. K., "On the optimality of the probability ranking scheme in storage applications," *J. Assoc. Comput. Mach.* **20** (1973), 624-633.